

3주차 · Gazebo VRX 구축과 좌표계

캡스톤디자인 2026-2 · 충남대학교 자율운항시스템공학과 | 갱신 2026-09-15

★ 참조 강의 — 본 과목이 전제하는 배경

아래 다섯 과목은 본 과목 담당 교수가 직접 강의한 것이며, 본 과목이 전제하는 배경 지식에 해당한다. 학부 기초에서 대학원 과정까지 이어지므로 부족한 지점부터 시작하면 된다. 본 문서에서 쓰는 좌표계·기호·유도 과정은 아래 강의에서 상세히 다루므로, 선수 지식이 부족한 경우 먼저 보고 돌아올 것.

#	과목	수준	언어	영상	자료·코드
1	제어공학 — 전달함수, 되먹임, 안정도, 근궤적, PID. 모든 것의 토대	학부	한국어	재생목록	드라이브
2	제어시스템설계 — 해석이 아니라 설계. 사양 결정, 루프 정형화, 이산 구현	학부	한국어	재생목록	드라이브
3	캡스톤디자인 — 본 과목의 지난 학기 강의. 이동체 프로젝트를 처음부터 끝까지	학부	한국어	재생목록	드라이브
4	제어공학특론 — 좌표계, 6자유도 운동방정식, 회전행렬과 오일러각, 선형화와 트림	대학원	영어	재생목록	드라이브
5	센서신호처리 및 융합 — 센서 모델, 잡음, 추정, 다중센서 융합	대학원	영어	재생목록	드라이브

MATLAB-Simulink 가 처음이라면 6주차 실습 전에 아래를 끝낼 것. 본 과목의 제어가 실습은 전부 Simulink 로 진행한다. Onramp 는 무료이며 각각 몇 시간이면 끝난다.

도구	시작 지점
MATLAB	MATLAB Onramp · Core MATLAB Skills
Simulink	Simulink Onramp · 담당 교수 Simulink 강의 1부 · 2부

- 과목: 캡스톤디자인 (2026-2) · 충남대학교 자율운항시스템공학과
- 이번 주차 학습 내용: ① 시뮬레이터 설치 ② 배를 물에 띄우고 직접 조종 ③ 좌표계 정리

★ 시작 전 확인

- 2주차 ROS 2 설치가 끝나 있어야 함
- `ros2 run demo_nodes_cpp talker` 가 되는지 먼저 확인할 것
- 저장공간 20 GB 이상: `df -h` 로 확인
- 빌드에 30~60분 걸림. 충전기를 꽂고, 절전 모드로 두지 말 것

이 주차를 마치면 할 수 있어야 하는 것

1. SDF · URDF · Xacro 의 역할 차이 설명
2. Gazebo Garden + VRX 설치하고 WAM-V를 물에 띄우기
3. 명령어로 배를 직진 · 선회시키기
4. 차동 추진이 무엇인지 설명하고, 키보드로 배를 조종하기

5. 선박 6자유도와 ENU / NED / Body 좌표계 구분
6. 쿼터니언 ↔ 오일러각 변환 이해
7. TF2 트리 읽기

준비물

항목	내용
환경	2주차에 만든 ROS 2 Humble (<code>ros2 topic list</code> 가 동작해야 함)
저장공간	20 GB 이상 — Gazebo Garden + VRX 빌드에 필요 (실측 <code>vrx_ws</code> 904 MB + 의존 패키지)
전원	충전기 지참 . 노트북에서는 빌드가 10분 이상 걸릴 수 있다
그래픽	GUI 가 뜨는지 1주차 2-4절로 미리 확인
실습 코드	2주차 <code>usv_basics</code> 저장소 — https://github.com/wkyouncnu/usv_basics

✕ 이번 주차에는 빌드 시간이 길다

`colcon build` 를 걸어 놓고 그동안 1부 이론(좌표계)을 듣는 순서로 진행한다.
수업 시작하자마자 빌드를 먼저 건다.

1부 · 이론

1-1. 물리 시뮬레이터는 무엇을 계산하는가

매 시간 스텝마다 반복되는 일

1. 외력 계산
 - 추진기 플러그인 -> 추력
 - 유체력 플러그인 -> 부가질량, 감쇠, 부력
 - 파랑 플러그인 -> 파력
 - 바람 플러그인 -> 풍력
2. 강체 동역학 적분
 - 힘 = 질량 x 가속도 -> 새로운 위치 · 자세 · 속도
3. 센서 시뮬레이션
 - GNSS : 참값 + 잡음
 - IMU : 참값 + 바이어스 + 잡음
 - LiDAR : 레이캐스팅 -> 포인트클라우드
 - 카메라: 렌더링 -> 영상
4. ROS 2 토픽으로 발행

- 기본 스텝: 1 ms

! 본 제어기는 이 루프의 밖에 있음

- Simulink나 Python 노드가 토픽으로 상태를 받아 명령을 돌려줌
- Gazebo가 그것을 다음 스텝의 추력으로 사용
- 즉 **협조 시뮬레이션**이지, 한 스텝씩 맞물려 도는 방식이 아님

시뮬레이션 시간과 RTF

용어	뜻
sim time	시뮬레이터 안의 시간. 벽시계 시간과 다름
RTF (Real Time Factor)	시뮬레이션 시간 ÷ 실제 시간

- RTF = 1.0 → 실시간과 같은 속도
- VRX + 무거운 센서 → 보통 **0.3 ~ 0.7**
- Gazebo 창 하단에 표시됨

⚠ 알고리즘의 오류가 아니라 처리 속도 문제일 수 있다

노트북 성능이 낮으면 RTF가 떨어짐. **항상 RTF를 먼저 확인할 것.**

1-2. SDF · URDF · Xacro

형식	정체	쓰임
SDF	Gazebo 전용 XML	월드 전체 — 바다, 부표, 도킹 스테이션, 조명
URDF	ROS 표준 로봇 기술 XML	로봇 한 대 — 링크, 조인트, 센서
Xacro	URDF를 만들어 내는 매크로	반복 제거, 값 대입

왜 Xacro가 필요한가

- 추진기 · 센서는 위치만 다르고 구조는 같음
- Xacro 없이 쓰면 같은 XML 블록을 개수만큼 복사해야 함
- 실제 파일 `vrx_urdf/wamv_gazebo/urdf/thruster_layouts/wamv_aft_thrusters.xacro`
 - 본 과목에서 쓰는 기본 구성이다

```
<xacro:include filename="$(find wamv_description)/urdf/thrusters/engine.xacro" />
<xacro:engine prefix="left" position="-2.373776 1.027135 0.318237" />
<xacro:engine prefix="right" position="-2.373776 -1.027135 0.318237" />
```

- `xacro:engine` 매크로 하나를 위치만 바꿔 두 번 호출
- 매크로 없이 쓰면 같은 XML 블록을 두 번 복사해야 함
- 추진기가 4개, 6개로 늘어나면 차이가 커짐

★ 본 과목은 후방 2추진기 기본 모델을 쓴다

- 좌 · 우 각 1개, 선미($x = -2.374$ m)에 좌우로 ± 1.027 m 떨어져 있음
- 장착각을 지정하지 않았으므로 **선수 방향으로 고정**
- 좌우 추력 차이로 방향을 바꾸는 **차동 추진(differential thrust)**
- 3주차 실습에서 이미 이렇게 조종해 봤음

- 변환 흐름

```
component_config.yaml └─
thruster_config       └─> Xacro 처리 -> URDF -> SDF -> Gazebo에 스폰
wamv_gazebo.urdf.xacro └─
```

- 실제 파일은 4주차에 열어 봄
 - `src/vrx/vrx_urdf/wamv_gazebo/urdf/thruster_layouts/wamv_aft_thrusters.xacro`

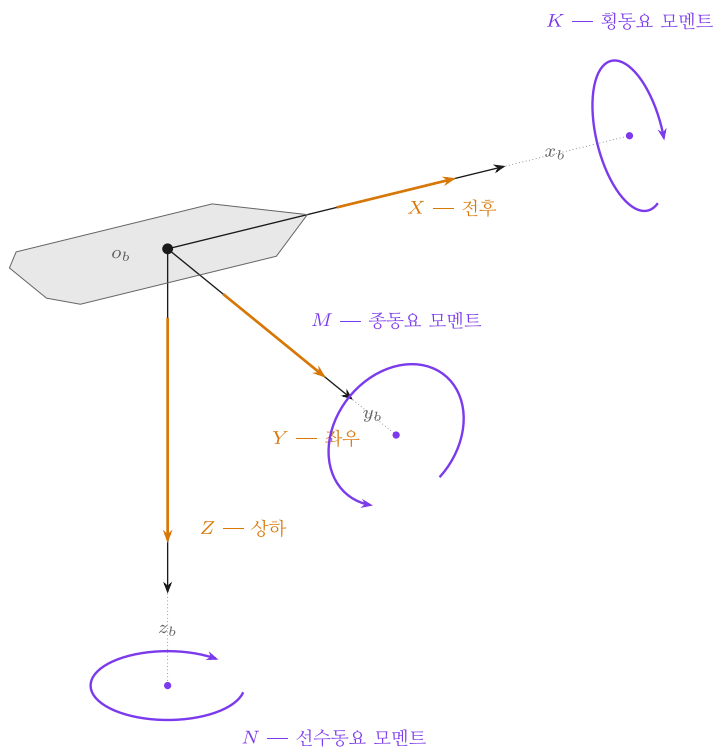
1-3. VRX 플러그인 — 무엇이 배를 움직이는가

플러그인	역할	조절 가능한 것
Hydrodynamics	부가질량, 감쇠	유체력 계수
Buoyancy	부력	배수량, 흘수
Wavefield	파랑 생성·파력	파고, 주기, 방향
Wind	풍력	풍속, 방향
Thruster	추력 생성	추력계수, 최대 명령
Joint Position Controller	추진기 틸트각 (본 과목에서는 0 고정, 미사용)	각도 명령
센서	측정값 + 잡음	갱신주기, 잡음 크기

! 왜 알아야 하는가

14주차에 "파랑 3단계, 바람 3단계" 조건으로 반복시험을 함.
그때 조절하는 것이 바로 이 파라미터들임.

1-4. 선박의 6자유도 운동



선박의 6자유도

한 줄이 곧 하나의 인과다. 가로로 읽는다 — 힘을 주면 속도가 생기고, 그 속도를 적분하면 위치가 된다.

#	자유도	힘 · 모멘트	속도	위치 · 자세
1	전후동요 (surge)	X	u	x
2	좌우동요 (sway)	Y	v	y
3	상하동요 (heave)	Z	w	z
4	횡동요 (roll)	K	p	ϕ
5	종동요 (pitch)	M	q	θ
6	선수동요 (yaw)	N	r	ψ

직선 화살표 = 힘. 곡선 화살표 = 모멘트이며, 자기 축에 수직인 평면 위의 원으로 그린다 (오른손 법칙). 점선은 각 원의 중심을 자기 축에 잇는 것으로, 중심이 축에서 벗어난 타원은 이 그림이 틀리는 전형적인 방식이다.

수상선은 보통 **1 · 2 · 6**(전후 · 좌우 · 선수각)으로 줄여 다룬다. 나머지 셋은 제어 대상에서 뺄 뿐 사라지지 않는다 — 파도가 커지면 Roll · Pitch 가 커지고 그만큼 측정값에 잡음이 섞인다.

이 표는 가로로 읽는다

한 줄이 곧 하나의 인과다 — 힘을 주면 속도가 생기고, 그 속도를 적분하면 위치가 된다.
세로로 기호만 외우면 나중에 쓸 때 다시 찾게 된다.

#	자유도	이름	힘 · 모멘트	속도	위치 · 자세	뜻
1	전후동요	Surge	X	u	x	앞뒤로 나아감
2	좌우동요	Sway	Y	v	y	옆으로 미끄러짐
3	상하동요	Heave	Z	w	z	위아래로 오르내림
4	횡동요	Roll	K	p	ϕ	좌우로 기울어짐
5	종동요	Pitch	M	q	θ	앞뒤로 기울어짐
6	선수동요	Yaw	N	r	ψ	벡머리 방향이 돌아감

i "선수각" 과 "선수동요" 는 다른 말이다

선수동요(yaw)는 6번째 자유도의 이름이고, 선수각 ψ 는 그 자유도의 각이다.
같은 줄의 N 은 요 모멘트(N·m), r 은 요 각속도(rad/s)다. 한 줄에 네 가지가 있다.

N 도 두 가지로 쓰인다. 이 표의 N 은 요 모멘트이고, NED 의 N 은 북쪽 위치다.
본 과목은 둘 다 관례대로 N 으로 쓴다. 식을 보면 구분된다 — 위치와 함께 나오면 ($N - N_k, \text{atan2}(E, N)$) 북쪽, 힘과 함께 나오면 ($X, Y, N, F_L - F_R$) 모멘트다.
대학원 강의는 이 혼동을 피하려고 위치를 x^n, y^n 으로 쓴다.

- 힘·모멘트 기호 X, Y, Z, K, M, N 은 10주차 추력 배분에서 그대로 쓴다.
거기서 $\tau = [X, Y, N]^T$ 로 묶이는 것이 이 표의 1·2·6행이다

3자유도 근사

- 수상선의 수평면 운동은 보통 Surge · Sway · Yaw 3자유도로 다룸

$$\eta = [x, y, \psi]^T \quad \text{위치와 선수각 (지구 고정, NED)}$$

$$\nu = [u, v, r]^T \quad \text{속도 (선체 고정, Body)}$$

★ 위치는 지구 축, 속도는 배 축

η 와 ν 가 다른 좌표계라는 점이 이 표기의 핵심이다.
두 벡터를 잇는 것이 아래 1-5 의 변환이다.

- Heave · Roll · Pitch 는 파랑에 의한 진동으로 보고 제어 대상에서 제외

i 뺐다고 없어지는 것은 아니다

제어 대상에서 뺄 뿐, 배는 여전히 그 방향으로 움직인다.
파도가 커지면 Roll·Pitch 가 커지고, 그만큼 Surge·Sway 측정값에 잡음이 섞인다.
8주차에서 거친 월드(wayfinding2)로 바뀌 보면 이 영향이 눈에 보인다.

- 7주차에 Fossen 운동방정식으로 정식화함

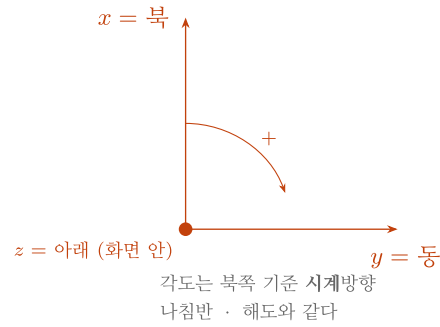
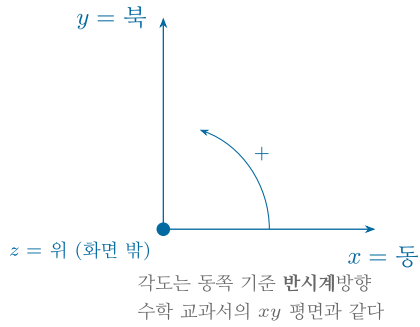
1-5. 좌표계 — 본 과목에서 가장 혼동하기 쉬운 부분

ENU

ROS · Gazebo 가 쓰는 것

NED

선박 · 제어 이론이 쓰는 것



$$N = y_{\text{ENU}}, \quad E = x_{\text{ENU}}, \quad D = -z_{\text{ENU}}$$

$$\psi_{\text{NED}} = 90^\circ - \psi_{\text{ENU}}$$

두 규약을 섞으면 오류 메시지 없이 배가 엉뚱한 방향으로 간다. 변환하는 지점을 한 곳으로 모으고 그 자리에 검사를 둔다

세 개의 좌표계

좌표계	축	각도 기준	누가 쓰는가
ENU	x=동, y=북, z=위	동쪽 기준 반시계	ROS · Gazebo
NED	x=북, y=동, z=아래	북쪽 기준 시계	선박 · 항공 제어
Body	x=선수, y=우현, z=아래	—	배와 함께 움직임

× 되돌릴 수 없는 조작

- VRX(ROS/Gazebo)는 **ENU** 로 데이터를 줌
- 본 과목 제어 이론과 Simulink 모델은 **NED** 기준임
- 변환을 빠뜨리면 배가 반대로 돌거나 90도 어긋난 방향으로 감
- 그런데 **에러가 나지 않음**. 동작만 예상과 달라진일 뿐임

변환 공식

위치

$$N = y_{\text{ENU}}, \quad E = x_{\text{ENU}}, \quad D = -z_{\text{ENU}}$$

- 이 변환은 자기 자신이 역변환임. NED → ENU 도 같은 식을 씀

방위각

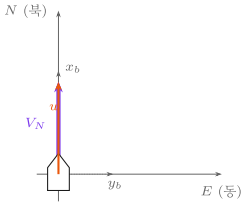
$$\psi_{\text{NED}} = 90^\circ - \psi_{\text{ENU}} \quad (\text{그 뒤 } -180^\circ \sim +180^\circ \text{ 로 정리})$$

각속도

$$\dot{r}_{\text{NED}} = -\dot{r}_{\text{ENU}} \quad (z\text{축 방향이 반대이므로 부호가 뒤집힘})$$

속도는 어느 축에서 잰 것인가 — $u \cdot v$ 와 $V_N \cdot V_E$

(1) $\psi = 0^\circ$ — 북쪽으로
선수가 북쪽, u 가 그대로 V_N 이라 두 화살표가
포개진다



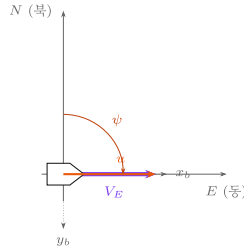
$$u = 1 \quad v = 0$$

$$V_N = 1 \quad V_E = 0$$

$$V_N = u \cos \psi - v \sin \psi$$

$$V_E = u \sin \psi + v \cos \psi$$

(2) $\psi = 90^\circ$ — 동쪽으로
배를 돌렸을 뿐인데 V_N 이 0 이 되었다. u 는
그대로 1.00

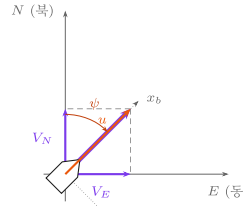


$$u = 1 \quad v = 0$$

$$V_N = 0 \quad V_E = 1$$

u, v 는 배에 붙어 있는 축에서 잰 속도다. 배를 돌려도 값이 그대로다.
 V_N, V_E 는 땅에 붙어 있는 축에서 잰 같은 속도다. 배를 돌리면 값이
바뀐다. (1)(2)(3) 은 u 가 모두 1.00 인데 V_N, V_E 만 바뀐다

(3) $\psi = 45^\circ$ — 북동으로
 u 하나가 두 성분으로 나뉜다

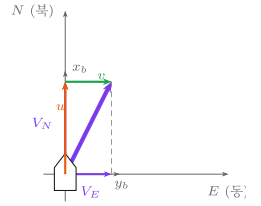


$$u = 1 \quad v = 0$$

$$V_N = 0.71 \quad V_E = 0.71$$

(4) 가 핵심이다. 선수각 ψ 는 배가 어디를 보는가이고, $\text{atan2}(V_E, V_N)$
은 배가 어디로 가는가이다. 옆으로 밀리면 둘이 달라진다. 그 차이가 크
랭각 $\beta = \text{atan2}(v, u)$ 다 (7주차)

(4) $\psi = 0^\circ$ 인데 옆으로 밀림
선수는 북쪽인데 북동으로 간다



$$u = 1 \quad v = 0.5$$

$$V_N = 1 \quad V_E = 0.5$$

같은 속도 하나를 두 좌표계로 읽은 것이다. 기호가 넷이라 다른 값처럼 보이지만
화살표는 하나다.

기호	어느 축에서 재는가	배를 돌리면
u (서지)	Body — 선수 방향	값이 그대로
v (스웨이)	Body — 우현 방향	값이 그대로
V_N	NED — 북쪽 성분	값이 바뀜
V_E	NED — 동쪽 성분	값이 바뀜

$$V_N = u \cos \psi - v \sin \psi$$

$$V_E = u \sin \psi + v \cos \psi$$

그림의 네 칸이 말하는 것.

칸	조건	u	v	V_N	V_E	읽는 법
(1)	$\psi = 0^\circ$	1.00	0	1.00	0	북쪽을 보고 북쪽으로
(2)	$\psi = 90^\circ$	1.00	0	0	1.00	u 는 그대로인데 V_N 이 0
(3)	$\psi = 45^\circ$	1.00	0	0.71	0.71	u 하나가 두 성분으로
(4)	$\psi = 0^\circ$, 옆으로 밀림	1.00	0.50	1.00	0.50	북쪽을 보는데 북동으로 간다

- (1)(2)(3) 은 u 가 모두 1.00 이다. 바뀐 것은 선수각뿐인데 V_N, V_E 가 달라진다
- 거꾸로 (4) 는 선수각이 (1) 과 같은데 V_E 가 생겼다. 배가 옆으로 밀렸기 때문이다

★ 선수각과 침로각은 다르다

- 선수각 ψ — 배가 어디를 보는가. 쿼터니언에서 나온다
- 침로각 $\chi = \text{atan2}(V_E, V_N)$ — 배가 어디로 가는가
- 둘의 차이가 크랭각 $\beta = \text{atan2}(v, u)$ 이며 $\chi = \psi + \beta$ 다
- 옆바람이나 조류가 있으면 $v \neq 0$ 이 되어 둘이 벌어진다.
7주차 LOS 유도에서 이 보정을 넣는다

✗ 속도 되먹임에 V_N, V_E 를 쓰지 않는다

속도 제어기는 u 를 되먹인다. V_N, V_E 는 배를 돌릴 때마다 값이 바뀌므로
게인이 따라다니지 못한다. u 는 배를 어느 쪽으로 돌리든 "앞으로 얼마나 빠르냐"
하나만 뜻한다.

잘못했을 때의 증상

빠뜨린 것	증상
x/y 교환	북쪽으로 가라고 했는데 동쪽으로 감
z 부호	좌선회 명령에 우선회함
각도 변환	헤딩이 90도 어긋난 채로 그럭저럭 따라감 ← 가장 위험

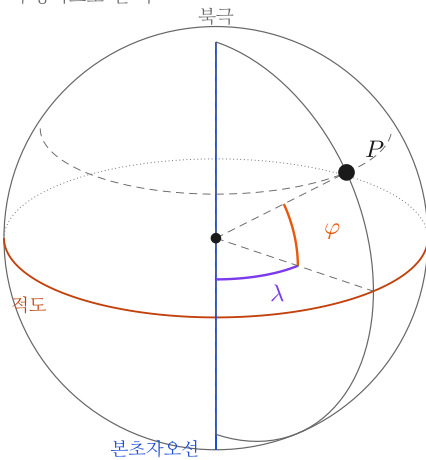
⚠ 부분적으로 동작하는 상태가 가장 위험하다

근사적으로 동작하므로 지나치기 쉬우나, 조건이 바뀌면 성능이 급격히 저하됨.
그래서 이번 주차 과제는 **수치로 증명**하게 함.

위경도를 미터로 — LLA → NED

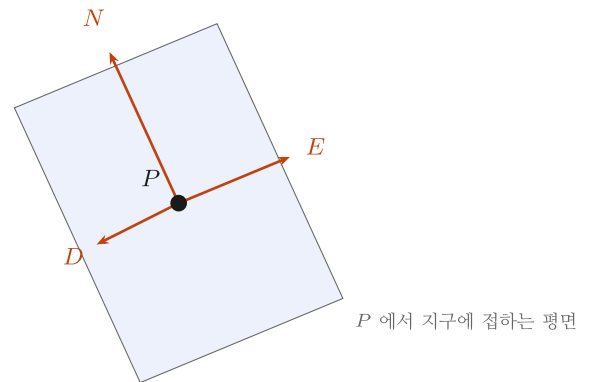
위도 φ 와 경도 λ

위도 는 적도에서 북쪽으로 잰 각, **경도** 는 본초자오선에 서 동쪽으로 잰 각



그 자리에 세우는 NED

N 은 자오선을 따라 북쪽, E 는 위도선을 따라 동쪽, D 는 지구 중심을 향한다



배가 수 km 안에서만 움직이면 이 평면을 평평하다고 보아도 된다. 그것이 flat-earth 근사이며, 위경도 차이에 곡률반경을 곱해 미터로 바꾼다

$$\Delta N = (\varphi - \varphi_0) R_M, \quad \Delta E = (\lambda - \lambda_0) R_N \cos \varphi_0, \quad \Delta D = -(h - h_0)$$

R_M 자오선 곡률반경, R_N 묘유선 곡률반경. 경도 쪽에 $\cos \varphi_0$ 가 붙는 이유는 위도가 높아질수록 경도 1도의 실제 거리가 짧아지기 때문이다 — 극에서는 0 이 된다

위도 φ 와 경도 λ 가 무엇인가

- 지구를 구로 보고, 중심에서 그 점을 향해 그은 선이 만드는 **두 각**이다
- **위도** — 적도면에서 북쪽으로 잰 각. 적도가 0° , 북극이 $+90^\circ$, 남극이 -90°
- **경도** — 영국 그리니치를 지나는 **본초자오선**에서 동쪽으로 잰 각. $-180^\circ \sim +180^\circ$
- 본 과목의 기준점은 남반구라 위도가 **음수**다 (-33.72°)

그 자리에 NED 를 세운다

- 그 점에서 지구에 **접하는 평면**을 하나 놓는다
- N — 자오선을 따라 북쪽. E — 위도선을 따라 동쪽. D — **지구 중심** 방향
- 배가 수 km 안에서만 움직이면 이 평면을 평평하다고 보아도 된다

★ 왜 경도에만 $\cos \varphi$ 가 붙는가

자오선(남북 방향)은 어느 위도에서나 같은 큰 원이다. 그래서 위도 1도의 거리는 어디서나 약 111 km 로 거의 같다.

위도선(동서 방향)은 극으로 갈수록 **작아지는 원**이다. 그 반지름이

$R \cos \varphi$ 이므로 경도 1도의 거리도 $\cos \varphi$ 배로 줄어든다.

적도에서 111 km 인 것이 위도 60° 에서는 절반, 극에서는 0 이 된다.

- GPS는 위도 · 경도 · 고도(LLA)를 줌
- 제어기는 미터 단위 지역 좌표가 필요
- **Flat-Earth 근사** (수 km 이내에서 유효)

$$\begin{aligned}\Delta N &= (\varphi - \varphi_0) R_M \\ \Delta E &= (\lambda - \lambda_0) R_N \cos \varphi_0 \\ \Delta D &= -(h - h_0)\end{aligned}$$

기호	뜻
φ, λ, h	위도 · 경도 · 고도
$\varphi_0, \lambda_0, h_0$	기준점의 위도 · 경도 · 고도
R_M, R_N	자오선 · 묘유선 곡률반경

- MATLAB에서는 한 줄

```
ned = lla2ned([lat lon alt], lla0, 'flat');
```

- 본 과목의 기준점 (시드니 레가타 해역)

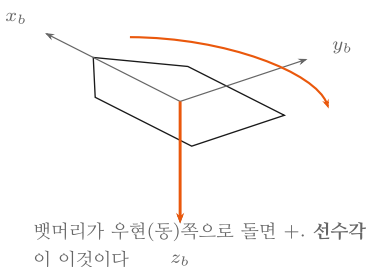
```
lla0 = [-33.72276870341191, 150.67399057896623, 1.183941401541233]
```

1-6. 회전 표현 — 오일러각 · 쿼터니언 · 회전행렬

① 오일러각 — 사람이 읽기 좋음

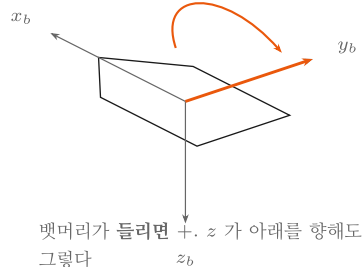
(1) 요 (Yaw)

ψ — z_b 둘레



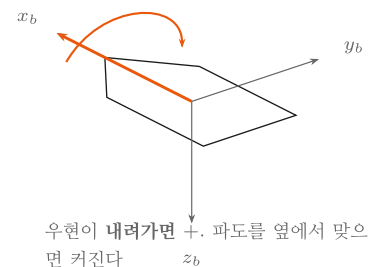
(2) 피치 (Pitch)

θ — 돌아간 y 축 둘레



(3) 롤 (Roll)

ϕ — 돌아간 x 축 둘레



세 번 돌린다 — 순서가 곧 정의다

$$\mathbf{R}(\phi, \theta, \psi) = \mathbf{R}_z(\psi) \mathbf{R}_y(\theta) \mathbf{R}_x(\phi)$$

요 → 피치 → 롤 순서로 돌린다. 둘째 · 셋째 회전은 이미 돌아간 축을 기준으로 한다. 순서를 바꾸면 다른 자세가 되므로, 오일러각은 "세 숫자" 가 아니라 "세 숫자와 순서" 다. 선박 · 항공은 z - y - x (요-피치-롤) 를 쓴다

피치가 $\pm 90^\circ$ 에 닿으면 롤 축과 요 축이 겹쳐 자유도 하나를 잃는다 (**짐벌락**). 수상선은 피치가 몇 도를 넘지 않아 실무에서 만나지 않지만, 저장 · 전송에는 이 문제가 없는 쿼터니언을 쓴다

오일러각은 축 하나를 골라 그 축으로 돌리기를 세 번 한 것이다.

순서	각	기호	도는 축	양(+)의 방향	배에서 보이는 모습
①	요 Yaw	ψ	z_b (아래)	벡머리가 우현(동) 쪽으로 돈다	벡머리 방향이 돌아감 — 선수 각
②	피치 Pitch	θ	요로 이미 돌아간 y 축	벡머리가 들린다	벡머리가 들렸다 잠겼다
③	롤 Roll	ϕ	요·피치로 이미 돌아간 x 축	우현이 내려간다	좌우로 기울어짐

✕ z 가 아래인데 피치 + 가 "벡머리 들림" 인 이유

오른손 법칙으로 y 축을 감으면 z 가 x 쪽으로 간다. z 가 아래이므로 x (벡머리)는 위로 간다. 처음 보면 틀린 것 같지만 맞다. 대학원 강의 W01 §1-2 와 Fossen 이 같은 규약이다.

순서가 곧 정의다. 선박 · 항공은 z - y - x 순서를 쓴다.

$$\mathbf{R}(\phi, \theta, \psi) = \mathbf{R}_z(\psi) \mathbf{R}_y(\theta) \mathbf{R}_x(\phi)$$

- 요 → 피치 → 롤 순서로 돌린다. 둘째·셋째 회전은 이미 돌아간 축을 기준으로 한다 — 피치는 요로 돌아간 y 축 둘레, 롤은 요·피치로 돌아간 x 축 둘레다
- 대학원 강의 W01 §1-3, Fossen, MSS **Rzyx** 가 모두 이 읽는 법이다

i 식을 오른쪽부터 읽어도 같은 자세가 나온다

$\mathbf{R}_z \mathbf{R}_y \mathbf{R}_x$ 를 오른쪽부터 읽으면 고정된 NED 축 둘레로 롤 → 피치 → 요 순서가 된다.

결과 자세는 똑같다. 다만 "움직이는 축을 따라 요부터" 가 배에서 실제로 느끼는 순서와 맞으므로 본 과목은 그쪽으로 설명한다. 둘을 섞어 "롤부터, 그런데 돌아간 축 기준" 으로 읽으면 틀린다.

- 순서를 바꾸면 다른 자세가 된다. 오일러각은 "세 숫자"가 아니라 "세 숫자와 순서"다
- 각 축의 회전행렬

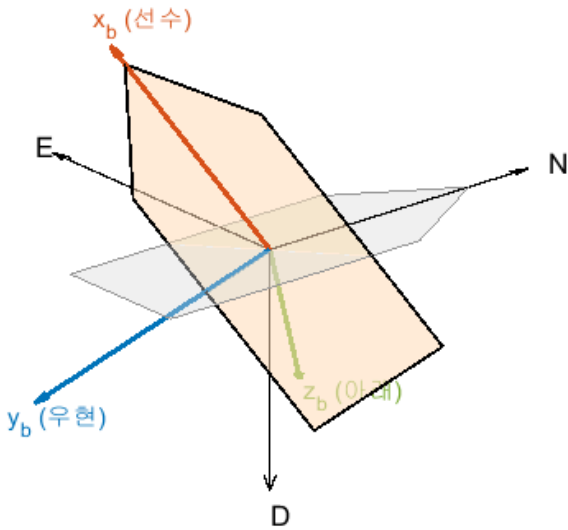
$$\mathbf{R}_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix}, \quad \mathbf{R}_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}, \quad \mathbf{R}_z(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

눈으로 확인 — 같은 세 숫자, 다른 자세

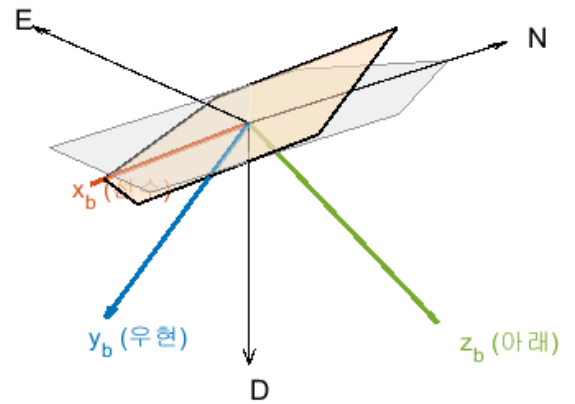
```
>> W03_euler_order
```

- 같은 세 각($\psi = 90^\circ$, $\theta = 30^\circ$, $\phi = 30^\circ$)을 순서만 바꿔 돌린다

z-y-x (선박 규약)
 $\psi=90^\circ, \theta=30^\circ, \phi=30^\circ$



x-y-z (순서만 바꿈)
 $\psi=90^\circ, \theta=30^\circ, \phi=30^\circ$



- 정상 출력 (MSS 가 경로에 있으면 **MSS Rzyx** 줄이 하나 더 나온다)

선수(x_b)가 가리키는 방향 [N E D]
 z-y-x (선박 규약) : [0.000 0.866 -0.500]
 x-y-z (순서 바꿈) : [0.000 0.866 0.500]
 두 선수 방향 사이의 각 : 60.0 deg
 MSS Rzyx 와의 차이 : 0.00e+00

선박 규약을 읽는 두 방법

움직이는 축 (요 -> 피치 -> 롤) 과 식의 차이 : 0.00e+00
 고정된 축 (롤 -> 피치 -> 요) 과 식의 차이 : 0.00e+00

읽는 법	뜻
선박 규약 $D = -0.500$	벋머리가 동쪽을 향하고 위로 30° 들렸다 — 피치 + 가 벋머리 들림이라는 것과 맞다
순서만 바꾸면 $D = +0.500$	같은 세 숫자인데 벋머리가 아래로 30° 숙였다. 두 선수 방향이 60° 벌어진다
MSS Rzyx 와 차이 0	본문의 식이 Fossen 툭박스와 같다
두 읽는 법 모두 차이 0	"움직이는 축으로 요부터" 와 "고정된 축으로 롤부터" 는 같은 행렬이다

짐벌락이란

- 피치가 $\pm 90^\circ$ 가 되면 롤 축과 요 축이 겹침
- 자유도 하나를 잃고, 변환식이 불안정해짐

! 수상선은 피치가 $\pm 90^\circ$ 가 될 일이 없음

그러면 배가 뒤집힌 것임. 그래서 실무상 오일러각으로 다루어도 됨.
 다만 저장과 전송은 쿼터니언이 표준이고, ROS 메시지도 전부 쿼터니언임.

② 쿼터니언 — 컴퓨터가 좋아함

쿼터니언은 "어느 축으로, 얼마나 돌렸는가" 를 네 숫자로 적은 것이다.
 회전축 단위벡터를 $\mathbf{n} = (n_x, n_y, n_z)$, 회전각을 α 라 하면

$$q = \left(\underbrace{\cos \frac{\alpha}{2}}_w, \underbrace{n_x \sin \frac{\alpha}{2}}_x, \underbrace{n_y \sin \frac{\alpha}{2}}_y, \underbrace{n_z \sin \frac{\alpha}{2}}_z \right), \quad w^2 + x^2 + y^2 + z^2 = 1$$

- 각을 **반으로** 나누어 넣는 것이 특징이다. 회전을 두 번 겹쳐 적용하는 대수 구조에서 나온다
- 크기가 항상 1 이다. 수치 오차로 1 에서 벗어나면 **정규화**한다
- q 와 $-q$ 는 **같은 자세**다. 부호가 뒤집혀 있어도 틀린 것이 아니다

간단한 예 — 요만 ψ 만큼 돌린 경우 (수상선이 거의 이 경우다)

$$\mathbf{n} = (0, 0, 1) \implies q = \left(\cos \frac{\psi}{2}, 0, 0, \sin \frac{\psi}{2} \right)$$

쿼터니언 → 오일러각

$$\begin{aligned} \phi &= \text{atan2}(2(wx + yz), 1 - 2(x^2 + y^2)) \\ \theta &= \arcsin(2(wy - zx)) \quad \leftarrow \text{이 값이 } \pm 1 \text{ 에 가까우면 짐벌락} \\ \psi &= \text{atan2}(2(wz + xy), 1 - 2(y^2 + z^2)) \end{aligned}$$

오일러각 → 쿼터니언

$$\begin{aligned} w &= c_\phi c_\theta c_\psi + s_\phi s_\theta s_\psi, & x &= s_\phi c_\theta c_\psi - c_\phi s_\theta s_\psi \\ y &= c_\phi s_\theta c_\psi + s_\phi c_\theta s_\psi, & z &= c_\phi c_\theta s_\psi - s_\phi s_\theta c_\psi \end{aligned}$$

여기서 $c_\phi = \cos \frac{\phi}{2}$, $s_\phi = \sin \frac{\phi}{2}$ 이며 θ, ψ 도 같다.

✗ 순서 관례가 두 가지임

- ROS · Gazebo: (x, y, z, w) — w 가 마지막
- MATLAB `quaternion` 객체: (w, x, y, z) — w 가 처음

이 불일치로 인한 버그가 매년 나옴.

변환 코드를 짤 때 **반드시 주석으로 순서를 명시**할 것.

🔪 본 과목에서 실제로 쓰는 것은 요 하나뿐이다

수평면 3자유도만 다루므로 $\phi \approx \theta \approx 0$ 이다. 그래서 위의 긴 식 대신 **요만 뽑는 한 줄**을 쓴다. 3주차 Simulink 모델과 6주차 `Quat2Yaw` 가 이것이다.

$$> \psi_{\text{ENU}} = \text{atan2}(2(wz + xy), 1 - 2(y^2 + z^2)) >$$

③ 회전행렬 (DCM)

- 3×3 행렬. 벡터를 한 좌표계에서 다른 좌표계로 옮김
- 수평면 3자유도에서 Body $\rightarrow$ NED

$$\mathbf{J}(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \end{bmatrix} = \mathbf{J}(\psi) \begin{bmatrix} u \\ v \\ r \end{bmatrix}$$

- 첫 두 줄을 풀어 쓰면 1-5 에서 본 그 식이다

$$\dot{x} = V_N = u \cos \psi - v \sin \psi, \quad \dot{y} = V_E = u \sin \psi + v \cos \psi$$

- $\mathbf{J}$ 는 **직교행렬**이라 역변환이 전치다 — $\mathbf{J}^{-1} = \mathbf{J}^T$.
NED 속도에서 몸체 속도로 돌아올 때 역행렬을 계산할 필요가 없다

정리

용도	권장 표현
저장 · 전송 (ROS 메시지)	쿼터니언
사람에게 보여주기, 로그	오일러각
벡터 좌표 변환	회전행렬

1-7. TF2 — 좌표계 관계를 관리하는 시스템

왜 필요한가

- 로봇에는 좌표계가 많음

```
map          지도 원점 (고정)
|
odom         주행거리계 원점
|
base_link    선체 중심
+-- lidar_link    LiDAR 장착 위치
+-- camera_link   카메라 장착 위치
+-- gps_link
+-- imu_link
```

- TF2가 푸는 문제
 - "LiDAR가 자기 앞 10 m에서 부표를 봤다. 그 부표는 지도상 어디인가?"

정적 TF vs 동적 TF

종류	토픽	예
정적 (static)	<code>/tf_static</code>	LiDAR ↔ 선체 (센서는 볼트로 고정됨)
동적 (dynamic)	<code>/tf</code>	선체 ↔ 지도 (배가 움직임)

실제 WAM-V 의 TF 트리 (실측)

```
ros2 run tf2_tools view_frames
```

- 5초간 듣고 현재 폴더에 `frames\_<날짜>.pdf` 를 만든다. 그 PDF 를 열면 트리 전체가 보인다
- 기준 환경 결과: **프레임 36개** (노트북 재측정 2026-09-15: 35개 - 발행 시점에 따라 1개 차이가 날 수 있음)

```
| 총 | 프레임 예 | 뜻 |
|---|---|---|
| 뿌리 | `wamv/wamv/base_link` | 배의 기준점. 모든 센서가 여기에 매달린다 |
| 기둥 | `..._post_link` → `..._post_arm_link` | 센서를 세우는 지지대. 실제 하드웨어 구조를 그대로 옮긴 것
|
| 센서 | `lidar_wamv_link`, `gps_wamv_link` | 센서 본체 |
| 광학 | `..._link_optical` | 카메라 전용. **광학 축(z 전방)** 으로 한 번 더 돌린 프레임 |
```

- `rate` 를 보면 성질이 드러난다

```
| rate | 성질 |
|---|---|
| 10000.0 | **정적 TF**. 한 번 발행하고 안 바뀐다 (센서 장착 위치) |
| 19.678 (노트북 19.447) | **동적 TF**. 매 순간 바뀐다 (추진기 회전 등). RTF 에 따라 조금씩 다름 |
| 250.214 | 센서 프레임(`.../imu_wamv_sensor`, `.../navsat` 등). 노트북 실측값 |
```

- > [!note] 왜 `optical` 프레임이 따로 있는가
 - > 로봇공학은 x 를 앞으로 보지만, 영상처리는 z 를 앞(광축)으로 본다.
 - > 두 관습이 충돌하므로 카메라마다 90° 씩 돌린 프레임을 하나 더 둔다.
 - > 12주차 영상처리에서 좌표가 안 맞으면 **이 프레임을 잘못 쓴 것**이 대부분이다.
- ### 명령어

```
```bash
ros2 run tf2_tools view_frames # 프레임 트리를 PDF로 저장
ros2 run tf2_ros tf2_echo 상위 하위 # 두 프레임 사이 변환을 실시간 출력
```

★ **frame\_id** 가 부정확하면 TF2 변환이 성립하지 않는다

2주차에 배운 **header.frame\_id** 가 여기서 값을 함.

## 2부 · 실습

### 2-1. Gazebo Garden 설치

#### 저장소 등록

```
sudo apt update
sudo apt install -y lsb-release curl gnupg
```

```
sudo curl https://packages.osrfoundation.org/gazebo.gpg \
 --output /usr/share/keyrings/pkgs-osrf-archive-keyring.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/pkgs-osrf-archive-keyring.gpg] http://packages.osrfoundation.org/gazebo/ubuntu-stable $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/gazebo-stable.list > /dev/null
```

⚠ 위 명령은 한 줄이다. 회색 상자 전체를 복사한다.

#### 설치

```
sudo apt update
sudo apt install -y gz-garden
```

#### ROS 2 연동 패키지

```
sudo apt install -y python3-sdformat13 ros-humble-ros-gz-garden ros-humble-xacro
```

#### 설치 확인

```
gz sim -v 4 shapes.sdf
```

- **정상:** 도형(구·상자·원기둥)이 놓인 3D 창이 뜸
- 처음 실행 시 모델 다운로드로 시간이 걸릴 수 있음
- 창이 안 뜨면 → 1주차 **xeyes** 확인으로 복귀
- 종료: 창을 닫거나 터미널에서 **Ctrl + C**

### 2-2. VRX 설치

#### 1단계 — 폴더 만들고 내려받기

```
mkdir -p ~/vrx_ws/src
cd ~/vrx_ws/src
git clone https://github.com/osrf/vrx.git
```

## 2단계 — 브랜치 지정 (가장 중요)

```
cd ~/vrx_ws/src/vrx
git checkout humble
```

- 저장소 <https://github.com/osrf/vrx> 를 열면 왼쪽 위 브랜치 표시가 **jazzy** 다. 내려받은 직후의 상태가 바로 이것이므로, 그대로 빌드하면 안 된다

### ✗ 생략할 수 없는 단계

- VRX의 기본 브랜치는 **최신 조합(ROS 2 Jazzy + Gazebo Harmonic)** 을 따라감
- 본 과목에서는 **Humble + Garden** 이므로 브랜치를 바꿔야 함
- 빠뜨리면 빌드가 실패하거나, 실행할 때 플러그인을 못 찾음
- 매년 가장 많이 나오는 실수임**

- 확인

```
git branch --show-current
```

- 정상 출력: **humble**
- 버전도 함께 확인해 둔다

```
git describe --tags
```

- 기준 환경 실측 — VRX **2.4.0-2** (2.4.1 준비 커밋 **dc30ed8d**)

```
2.4.0-2-gdc30ed8d
```

### i 기본 브랜치가 **jazzy** 라는 것은 이렇게 확인한다

```
git branch -r | grep 'origin/HEAD'
```

```
origin/HEAD -> origin/jazzy
```

### 3단계 — 의존성 설치

```
cd ~/vrx_ws
rosdep install --from-paths src --ignore-src -r -y
```

### 4단계 — 빌드

```
source /opt/ros/humble/setup.bash
colcon build --merge-install
```

#### ⚠ 빌드 도중 프로세스가 죽으면 메모리 부족이다

아래처럼 동시 작업 수를 줄여 다시 시도한다. 노트북을 절전 모드로 두지 말 것.

```
colcon build --merge-install --parallel-workers 2
```

- 정상 완료 출력 (기준 환경 실측, `--parallel-workers 2`)

```
Starting >>> vrx_gazebo
Starting >>> vrx_ros
Finished <<< vrx_gazebo [1.58s]
Starting >>> wamv_description
Finished <<< wamv_description [1.45s]
Starting >>> wamv_gazebo
Finished <<< wamv_gazebo [1.30s]
Finished <<< vrx_ros [16.5s]
Starting >>> vrx_gz
Finished <<< vrx_gz [23.0s]

Summary: 5 packages finished [39.7s]
```

항목	기준 환경 실측
패키지 수	5개 ( <code>vrx_gazebo</code> · <code>vrx_ros</code> · <code>wamv_description</code> · <code>wamv_gazebo</code> · <code>vrx_gz</code> )
빌드 시간	39.7초 (데스크톱) · 1분 54초 (Core Ultra 7 258V 노트북)
만들어진 플러그인	<code>install/lib/</code> 아래 <code>.so</code> 21개
워크스페이스 용량	904 MB

#### ! 빌드 시간은 노트북마다 크게 다르다

위는 최근 사양의 데스크톱 실측이다. 오래된 노트북에서는 10분 이상 걸릴 수 있다.  
중요한 것은 시간이 아니라 마지막 줄이 `5 packages finished` 인지다.

- 플러그인이 실제로 만들어졌는지 확인한다

```
ls ~/vrx_ws/install/lib/*.so | head -5
```

```
/home/cnu/vrx_ws/install/lib/libAcousticPerceptionScoringPlugin.so
/home/cnu/vrx_ws/install/lib/libAcousticPingerPlugin.so
/home/cnu/vrx_ws/install/lib/libAcousticTrackingScoringPlugin.so
/home/cnu/vrx_ws/install/lib/libBallShooterPlugin.so
/home/cnu/vrx_ws/install/lib/libGymkhanaScoringPlugin.so
```

## 5단계 — 환경 자동 적용

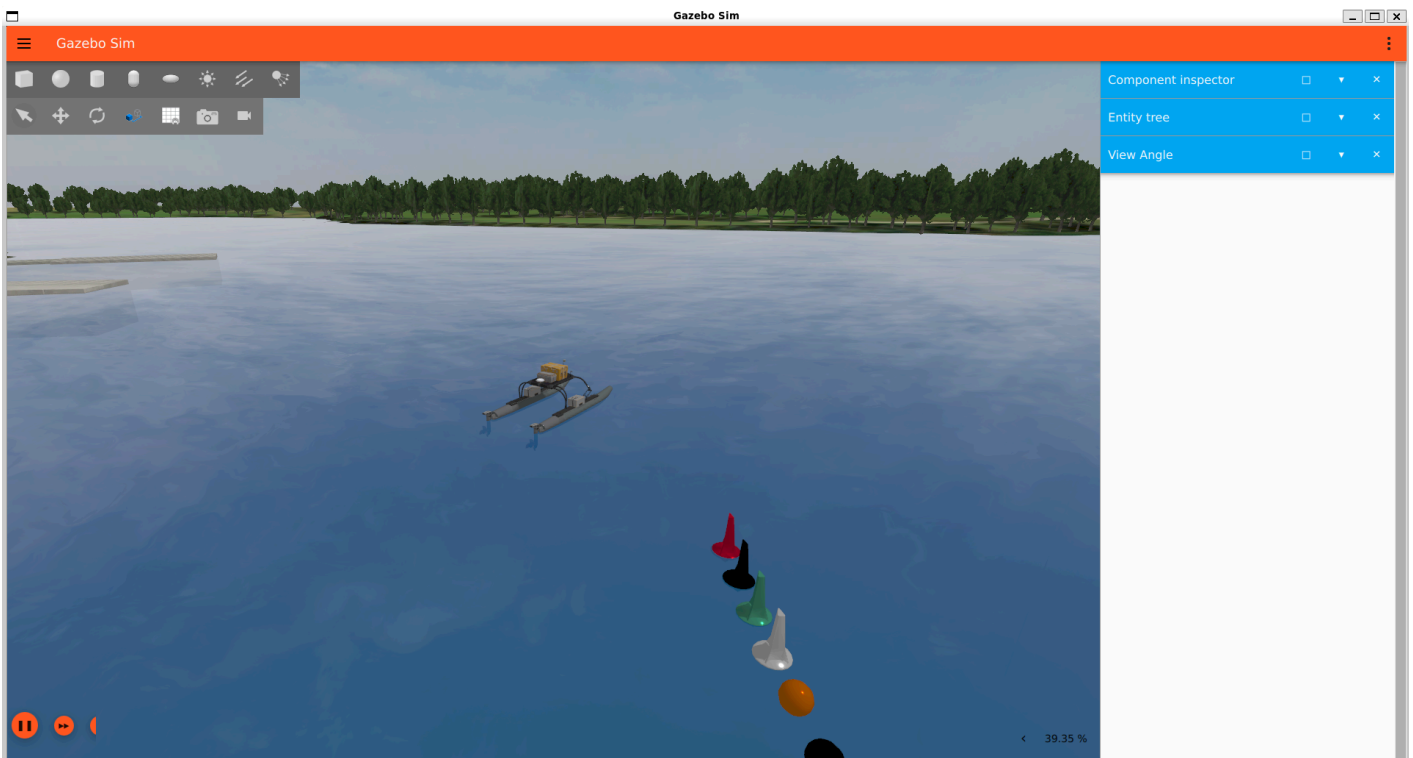
```
echo "source ~/vrx_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

### i 2주차 `capstone_ws` 와 함께 쓰기

`.bashrc` 에 두 줄이 다 있으면 됨. 나중에 적힌 쪽이 우선함.

## 2-3. 첫 실행 — 배를 물에 띄운다

### ★ 성공했을 때 이런 화면이 나온다



- 기준 환경에서 직접 실행해 캡처한 것이다 (VRX 2.4.1 · Gazebo Garden 7.9.0)
- 창이 뜨는 데 1~2분 걸린다. 검은 화면이 유지되어도 그동안은 정상이다

## 이 다섯 가지가 보이면 성공이다

#	확인 항목	화면에서
1	WAM-V 가 물에 떠 있다	가운데 회색 쌍동선
2	부표가 놓여 있다	빨강·검정·초록·흰색 원뿔과 주황 구
3	해안과 나무가 보인다	화면 위쪽. 시드니 레가타 센터
4	부두가 있다	왼쪽 위 회색 구조물
5	오른쪽 아래 RTF 가 0이 아니다	예: 37 % — 시간이 흐르고 있다는 뜻

### ⚠ 배가 가라앉거나 하늘로 솟구치면

물리 플러그인이 로드되지 않은 것이다. 터미널에서 **Hydrodynamics** 관련 오류를 찾는다.  
대개 `--force-version 7` 이 빠졌거나 Gazebo 버전이 섞인 경우다.

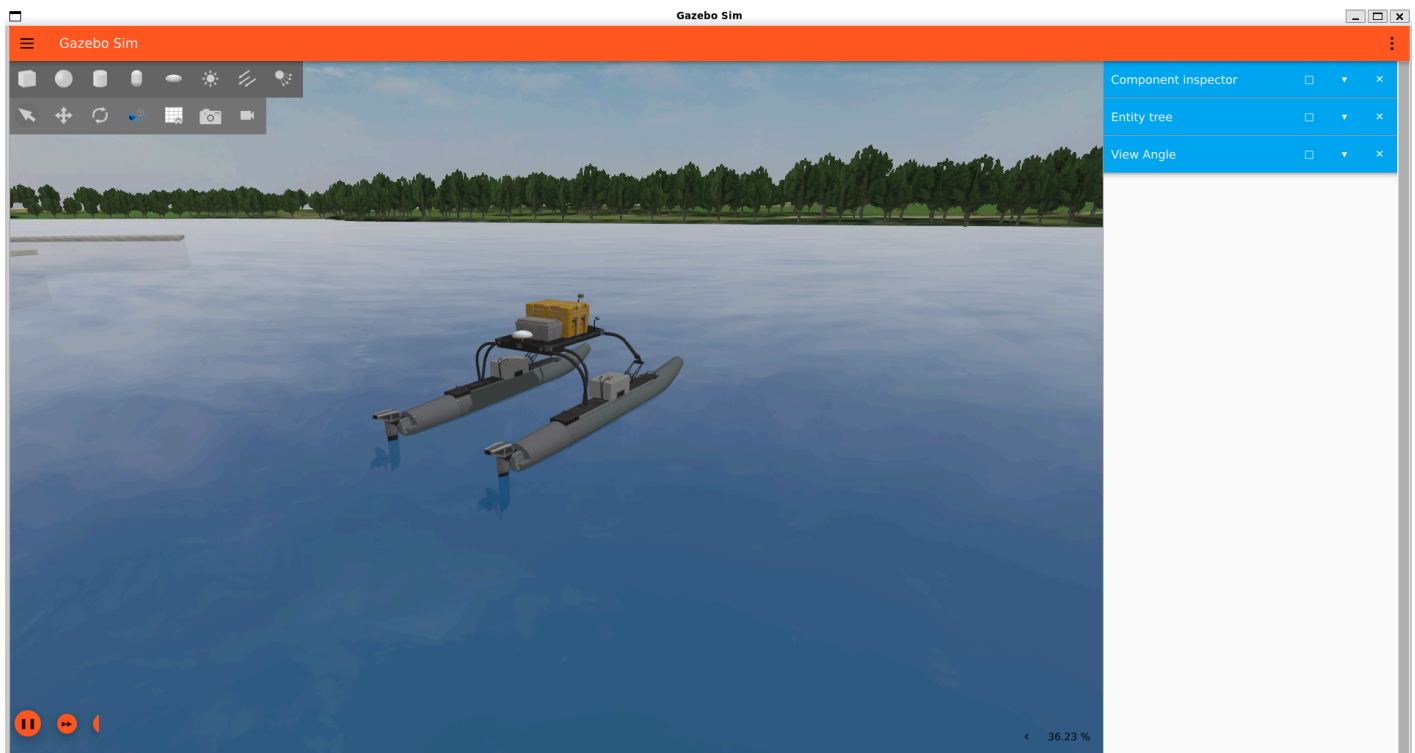
## 카메라를 배 가까이 가져가기

기본 시점은 배에서 멀다. 마우스로 옮겨도 되지만, 명령으로 하면 확실하다.

```
gz service -s /gui/follow --reqtype gz.msgs.StringMsg --reptype gz.msgs.Boolean --timeout 4000 --req 'data: "wamv"'
```

```
gz service -s /gui/follow/offset --reqtype gz.msgs.Vector3d --reptype gz.msgs.Boolean --timeout 4000 -req 'x: -7, y: -5, z: 3'
```

- 두 명령 모두 `data: true` 가 나오면 성공이다
- 카메라가 배를 따라다니며, 오프셋 숫자를 바꾸면 거리와 각도가 바뀐다



- 가까이서 보면 구조가 드러난다

보이는 것	설명
좌우로 나란한 원통 선체 두 개	쌍동선(catamaran). 그래서 좌우 추력 차이로 회전한다
뒤쪽 아래 프로펠러 두 개	후방 추진기. 4주차에서 위치를 실측한다
위쪽 노란 상자	배터리
마스트 위 흰 반구	GPS 안테나
마스트 앞 작은 상자들	카메라와 LiDAR

## 마우스 조작

동작	조작
회전	왼쪽 버튼 드래그
이동	가운데 버튼(휠) 드래그
확대·축소	휠 스크롤
물체 정보	왼쪽 버튼 클릭 → 오른쪽 <b>Component inspector</b>

## 실행 명령

```
ros2 launch vrx_gz competition.launch.py world:=sydney_regatta
```

- 시드니 레가타 해역에 WAM-V가 떠 있으면 성공

## RTF 가 1 % 미만이면 — 카메라 렌더링 병목

- 창은 떴는데 오른쪽 아래 실시간 계수가 **0.2 % ~ 1 %** 이면 배가 사실상 멈춘 상태임
- 이 상태에서는 §2-5-§2-6 에서 추력을 줘도 **움직임이 보이지 않음**
- 새 터미널에서 수치로 확인

```
gz topic -e -t /stats -n 1 | grep real_time_factor
```

- 비정상 출력 (Intel Arc 140V 노트북 실측)

```
real_time_factor: 0.0026878428978619109
```

- 원인: **센서 카메라 렌더링**. WSL 의 D3D12 그래픽 경로에서 **ogre2** 엔진의 카메라 3대가 극단적으로 느려짐
- 조치: 센서 렌더 엔진만 **ogre** 로 바꿔 실행. **월드·모델 파일은 고치지 않음**

```
ros2 launch vrx_gz competition.launch.py world:=sydney_regatta "extra_gz_args:=-render-engine-server ogre"
```

⚠ 위 명령은 한 줄이다. 따옴표까지 그대로 복사한다

- 정상 출력

```
real_time_factor: 0.98737029965454381
```

조건 (Intel Arc 140V · Core Ultra 7 258V, 2026-09-15 실측)	RTF
월드만 (배 없음)	99 %
배 + GPS · IMU · LiDAR (카메라 제외)	99 %
배 + 카메라 1대	0.85 %
기본 명령 (카메라 3대)	<b>0.26 %</b>
기본 + <code>--render-engine-server ogre</code>	<b>98 %</b> (GUI 포함 90 %)

- 판단 기준
  - RTF 가 **10 % 이상**이면 기본 명령 그대로 사용
  - RTF 가 **1 % 미만**이면 이후 모든 주차에서 위 옵션을 붙여 실행
- 카메라 토픽은 옵션을 붙여도 그대로 발행됨 ( `front_left_camera_sensor/image_raw` 수신 확인)

i 원인을 이렇게 좁혔다

- GUI 없이( `headless:=True` ) 실행해도 0.26 % → GUI 문제가 아님
- 소프트웨어 렌더링( `LIBGL_ALWAYS_SOFTWARE=1` )이 오히려 3배 빠름(0.85 %) → GPU 경유 렌더링 병목
- 센서를 하나씩 켜 보니 **카메라를 켜는 순간** 떨어짐 → 표의 결과

### 확인할 것 3가지

확인	의미
파도가 움직이는가	파랑 플러그인 동작 중
배가 파도를 따라 흔들리는가	유체력 · 부력 플러그인 동작 중
창 하단 RTF 값	1.0에 가까울수록 좋음. 0.3 이하면 무거운 것


**창이 표시되기까지 시간이 소요된다**

처음 실행하면 모델을 온라인에서 받음. 몇 분 기다릴 것.  
 받은 모델은 `~/.gz/fuel/` 에 저장되어 다음부터는 빠름.

## 2-4. 토픽 탐색

- 새 분할에서 실행 (VRX는 켜 둔 채)

```
ros2 topic list
```

- 수십 개가 나옴. 걸러서 봄

```
ros2 topic list | grep sensors
ros2 topic list | grep thrusters
```

### 센서 확인

```
ros2 topic echo /wamv/sensors/gps/gps/fix --once
```

```
ros2 topic hz /wamv/sensors/gps/gps/fix
```

```
ros2 topic echo /wamv/sensors/imu/imu/data --once
```

```
ros2 topic hz /wamv/sensors/lidars/lidar_wamv_sensor/points
```

✗ LiDAR 토픽에는 `echo` 를 사용하지 않는다

초당 수십만 개의 점이 글자로 쏟아져 터미널이 마비됨.

`hz` , `bw` , `info` 만 쓸 것.

## QoS 확인 — 2주차 복습

```
ros2 topic info /wamv/sensors/imu/imu/data --verbose
```

- `Reliability` 값을 적어 둘 것
- 4주차 과제(토픽 전수조사표)의 한 열이 됨

## 2-5. 배를 직접 움직여 본다

- 기본 WAM-V는 좌 · 우 2개 추진기를 가짐

### 직진

- 터미널 A

```
ros2 topic pub --rate 10 /wamv/thrusters/left/thrust std_msgs/msg/Float64 "{data: 200.0}"
```

- 터미널 B

```
ros2 topic pub --rate 10 /wamv/thrusters/right/thrust std_msgs/msg/Float64 "{data: 200.0}"
```

### 선회 (차동 추진)

- 좌현 약하게, 우현 강하게 → 좌선회

```
ros2 topic pub --rate 10 /wamv/thrusters/left/thrust std_msgs/msg/Float64 "{data: 50.0}"
```

```
ros2 topic pub --rate 10 /wamv/thrusters/right/thrust std_msgs/msg/Float64 "{data: 300.0}"
```

### 정지

- `data: 0.0` 으로 바꾸거나 `Ctrl + C`

### 관찰 과제 (필수)

관찰	질문
1	추력을 끊었을 때 얼마나 미끄러지는가?
2	선회 명령에 즉시 도는가, 지연이 있는가?
3	직진 명령만 줬는데 옆으로 흐르는가?

### ★ 이 세 가지가 왜 배 제어가 어려운지의 전부임

- 급정지가 안 됨 (관성)
- 응답이 느림
- 옆으로 흐름 (sway — 배는 자동차가 아님)

11주차에 충돌회피를 설계할 때 이 관찰을 반드시 기억할 것.

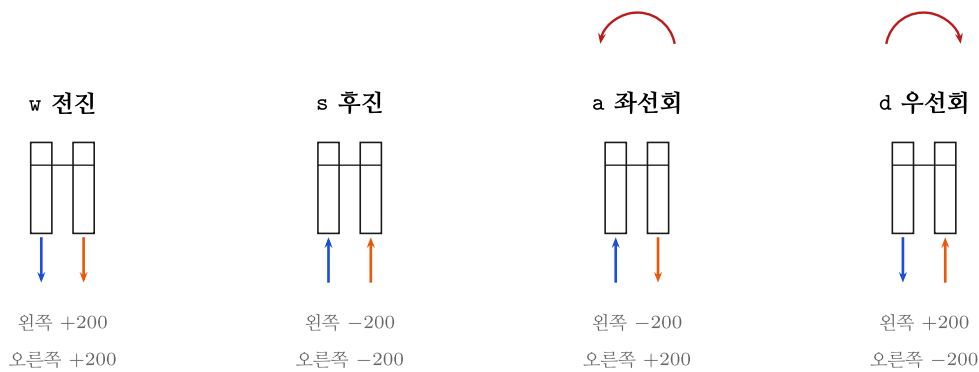
육상 로봇용 회피 알고리즘을 그대로 가져오면 실패하는 이유가 여기 있음.

## 2-6. 키보드로 배를 몬다 — `wamv_teleop_key`

### ★ 앞 절의 방식은 실습용으로 불편하다

`ros2 topic pub` 를 터미널 두 개에 띄워 놓고 값을 바꿔 치면 한 손으로 두 창을 오가야 한다. 노드 하나로 묶는다.

### 차동 추진 — 두 숫자로 배를 움직인다



WAM-V에는 조향타가 없다. 좌·우 추력 두 숫자만으로 전진·후진·선회를 만든다. 두 값이 같으면 직진, 다르면 선회 — 이것이 차동 추진이다

추진기 아래 화살표는 프로펠러가 물을 미는 방향이다. 배가 받는 힘은 그 반대다 — 물을 뒤로 밀면 배는 앞으로 간다

키	왼쪽 추력	오른쪽 추력	배의 움직임
w	+200	+200	전진
s	-200	-200	후진
a	-200	+200	좌선회 (제자리에서 왼쪽으로)
d	+200	-200	우선회
스페이스	0	0	정지

- 자동차와 다르다. 조향타가 없다. 좌·우 추력의 차이가 곧 선회다
- 그림의 화살표는 프로펠러가 물을 미는 방향이다. 배가 받는 힘은 반대다 — 전진(w)에서 화살표가 뒤를 향하는 이유
- 10주차에서 이 두 숫자를 추력 배분(thrust allocation)으로 자동 계산하게 된다

### 1단계 — 코드 받기

- 2주차에 받은 저장소에 노드가 추가되어 있다. 최신으로 갱신한다

```
cd ~/capstone_ws/src/usv_basics
git pull
```

- 정상 출력 (기준 환경 실측)

```
Fast-forward
 README.md | 69 ++++++-----
 setup.py | 1 +
 usv_basics/wamv_teleop_key.py | 138 ++++++-----
 3 files changed, 200 insertions(+), 8 deletions(-)
 create mode 100644 usv_basics/wamv_teleop_key.py
```

## ! 2주차 실습을 안 했으면 새로 받는다

```
mkdir -p ~/capstone_ws/src && cd ~/capstone_ws/src
git clone https://github.com/wkyouncnu/usv_basics.git
```

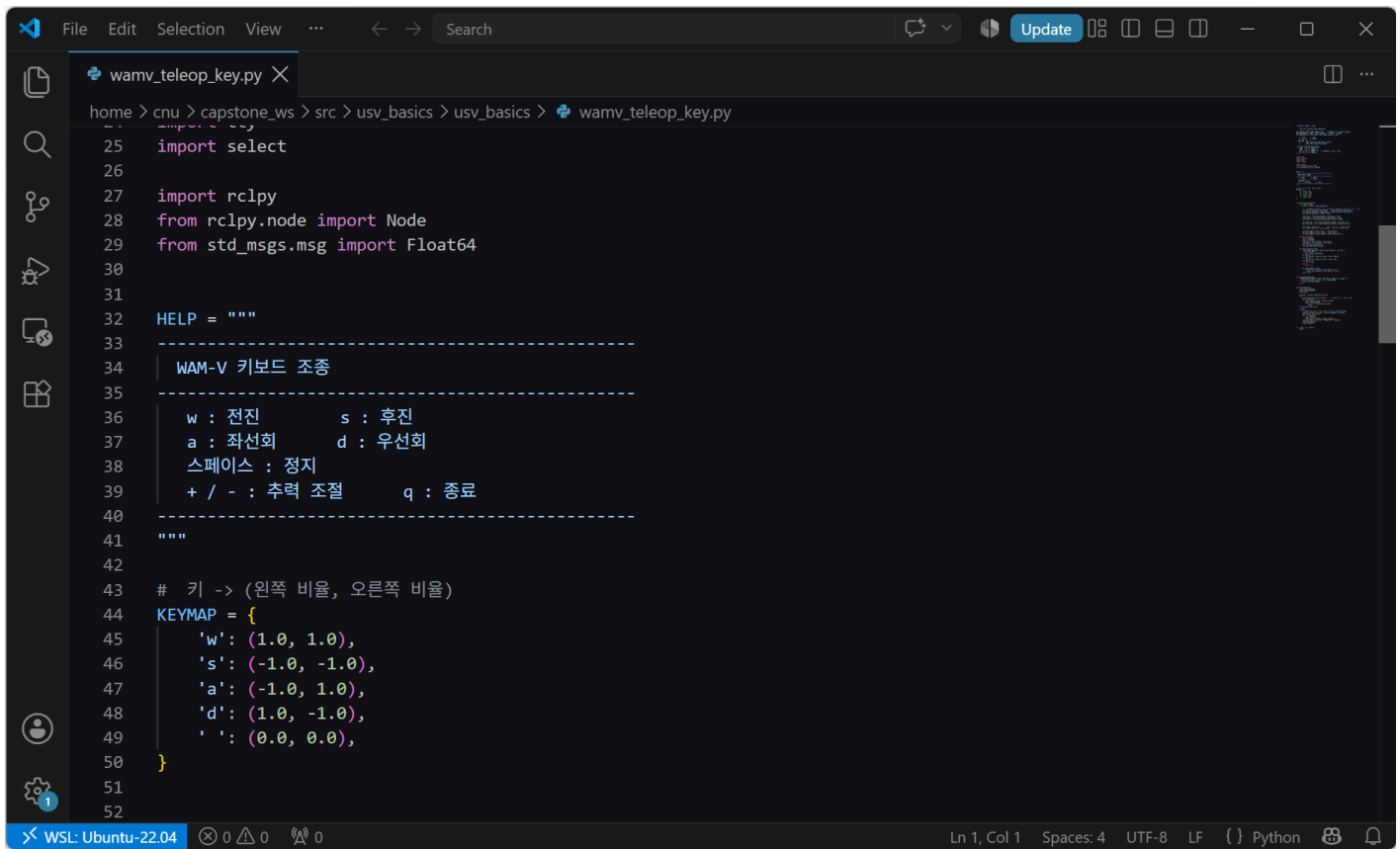
## 2단계 — 빌드

```
cd ~/capstone_ws
colcon build --symlink-install
source install/setup.bash
ros2 pkg executables usv_basics
```

- 정상 출력 — **wamv\_teleop\_key** 가 목록에 있어야 한다

```
usv_basics qos_test_pub
usv_basics qos_test_sub
usv_basics simple_listener
usv_basics simple_talker
usv_basics wamv_teleop_key
```

### 3단계 — 코드 읽기



```
25 import select
26
27 import rclpy
28 from rclpy.node import Node
29 from std_msgs.msg import Float64
30
31
32 HELP = """
33 -----
34 | WAM-V 키보드 조종
35 | -----
36 | w : 전진 s : 후진
37 | a : 좌선회 d : 우선회
38 | 스페이스 : 정지
39 | + / - : 추력 조절 q : 종료
40 | -----
41 | """
42
43 # 키 -> (왼쪽 비율, 오른쪽 비율)
44 KEYMAP = {
45 'w': (1.0, 1.0),
46 's': (-1.0, -1.0),
47 'a': (-1.0, 1.0),
48 'd': (1.0, -1.0),
49 ' ': (0.0, 0.0),
50 }
51
52
```

- 핵심은 **딕셔너리 하나다**. 키 → (왼쪽 비율, 오른쪽 비율)

```
KEYMAP = {
 'w': (1.0, 1.0),
 's': (-1.0, -1.0),
 'a': (-1.0, 1.0),
 'd': (1.0, -1.0),
 ' ': (0.0, 0.0),
}
```

- 그 비율에 추력을 곱해 **10 Hz 로 계속** 내보낸다

```
def on_timer(self):
 left = Float64()
 right = Float64()
 left.data = self.ratio[0] * self.thrust
 right.data = self.ratio[1] * self.thrust
 self.pub_left.publish(left)
 self.pub_right.publish(right)
```

왜 이렇게 했는가	이유
키를 눌러도 <b>계속 발행</b> 한다	추진기 플러그인은 마지막 값을 유지한다. 한 번만 보내면 계속 간다
종료( <b>q</b> ) 할 때 <b>0</b> 을 보낸다	안 보내면 창을 닫아도 배가 계속 나아간다
토픽 이름을 <b>파라미터</b> 로 뺐다	4주차에서 배 이름이 바뀌어도 코드를 안 고친다

- 엔터 없이 키 한 글자를 받기 위해 터미널을 **cbreak 모드**로 바꾼다

```
tty.setcbreak(sys.stdin.fileno())
```

### ⚠ 그래서 이 터미널에 포커스가 있어야 키가 먹는다

Gazebo 창을 클릭한 상태로 방향키를 눌러도 배는 움직이지 않는다.  
2주차 `turtle_teleop_key` 와 같은 이유다.

## 4단계 — 실행

- VRX 가 떠 있는 상태에서 새 터미널을 연다

```
ros2 run usv_basics wamv_teleop_key
```

- 정상 출력 (기준 환경 실측)

```
[INFO] [wamv_teleop_key]: left -> /wamv/thrusters/left/thrust
[INFO] [wamv_teleop_key]: right -> /wamv/thrusters/right/thrust
[INFO] [wamv_teleop_key]: thrust = 200 N
```

-----  
WAM-V 키보드 조종

-----  
w : 전진            s : 후진  
a : 좌선회        d : 우선회  
스페이스 : 정지  
+ / - : 추력 조절      q : 종료  
-----

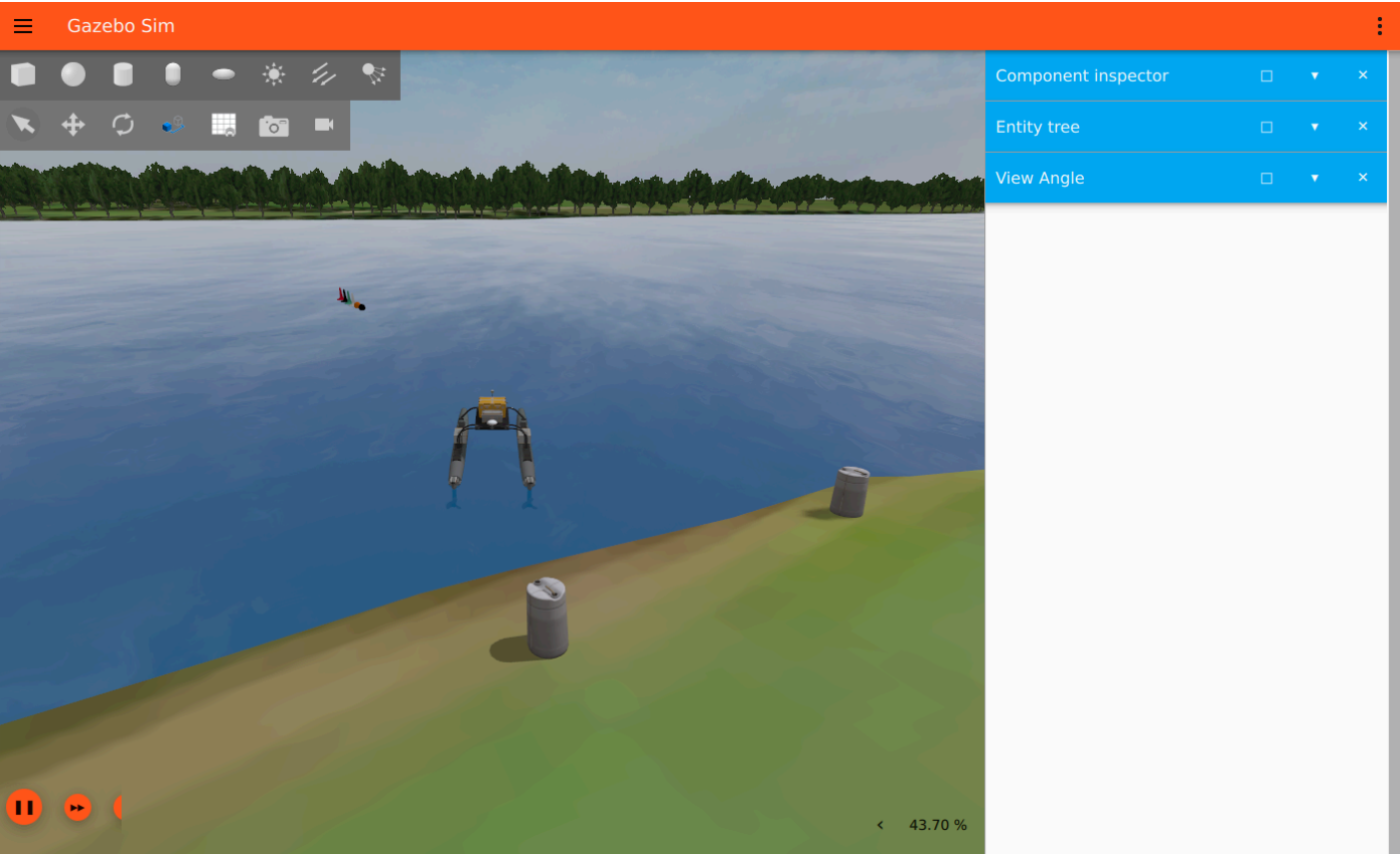
- 키를 누를 때마다 좌·우 추력이 찍힌다

```
[INFO] [wamv_teleop_key]: left 200.0 N right 200.0 N
[INFO] [wamv_teleop_key]: left -200.0 N right 200.0 N
[INFO] [wamv_teleop_key]: left 0.0 N right 0.0 N
[INFO] [wamv_teleop_key]: 정지 명령을 보내고 종료함
```

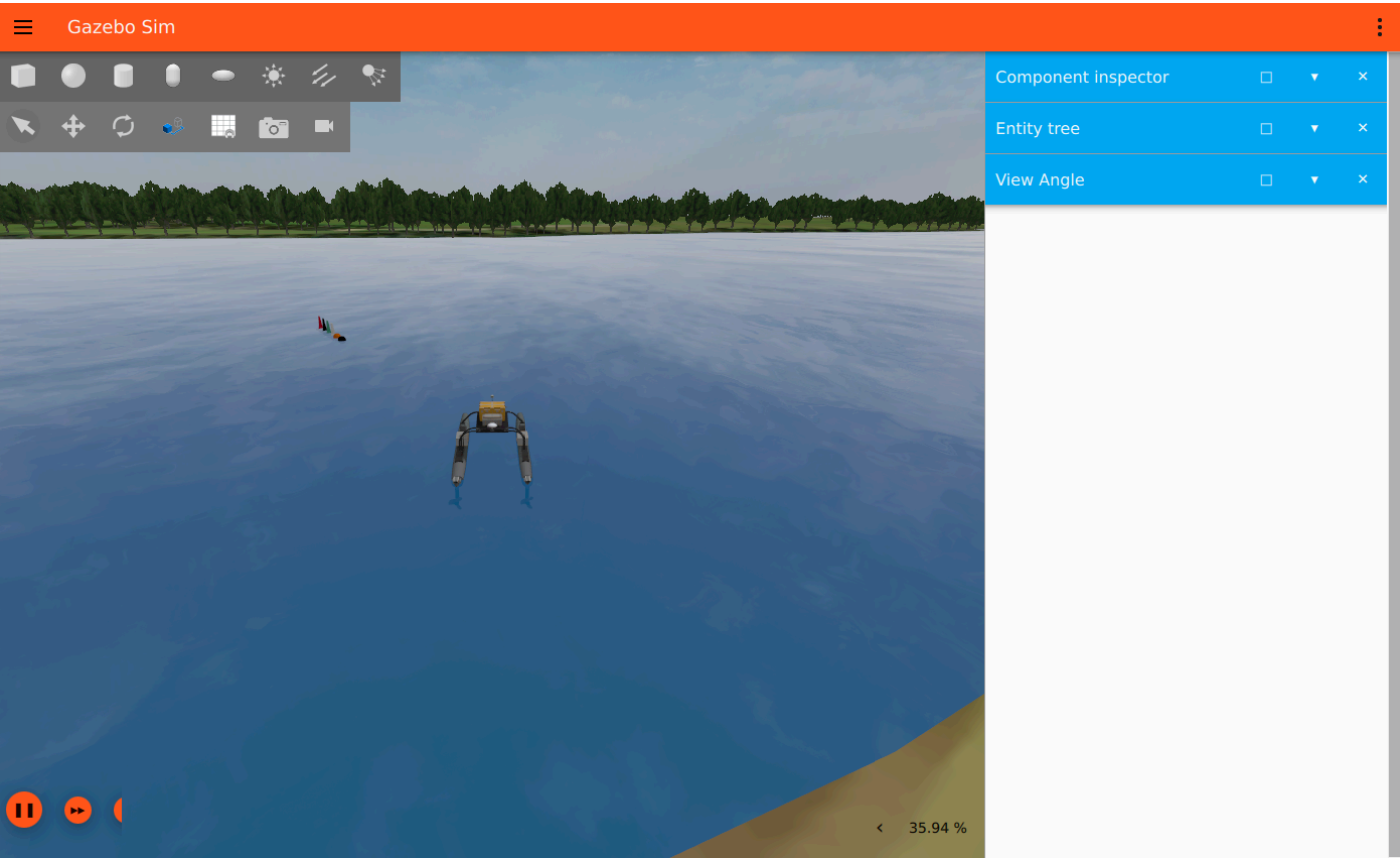
## 화면으로 확인 — 실제로 이렇게 움직인다

- 카메라를 배에 붙여 두면 따라다닌다 (아래 §카메라 고정 참조)

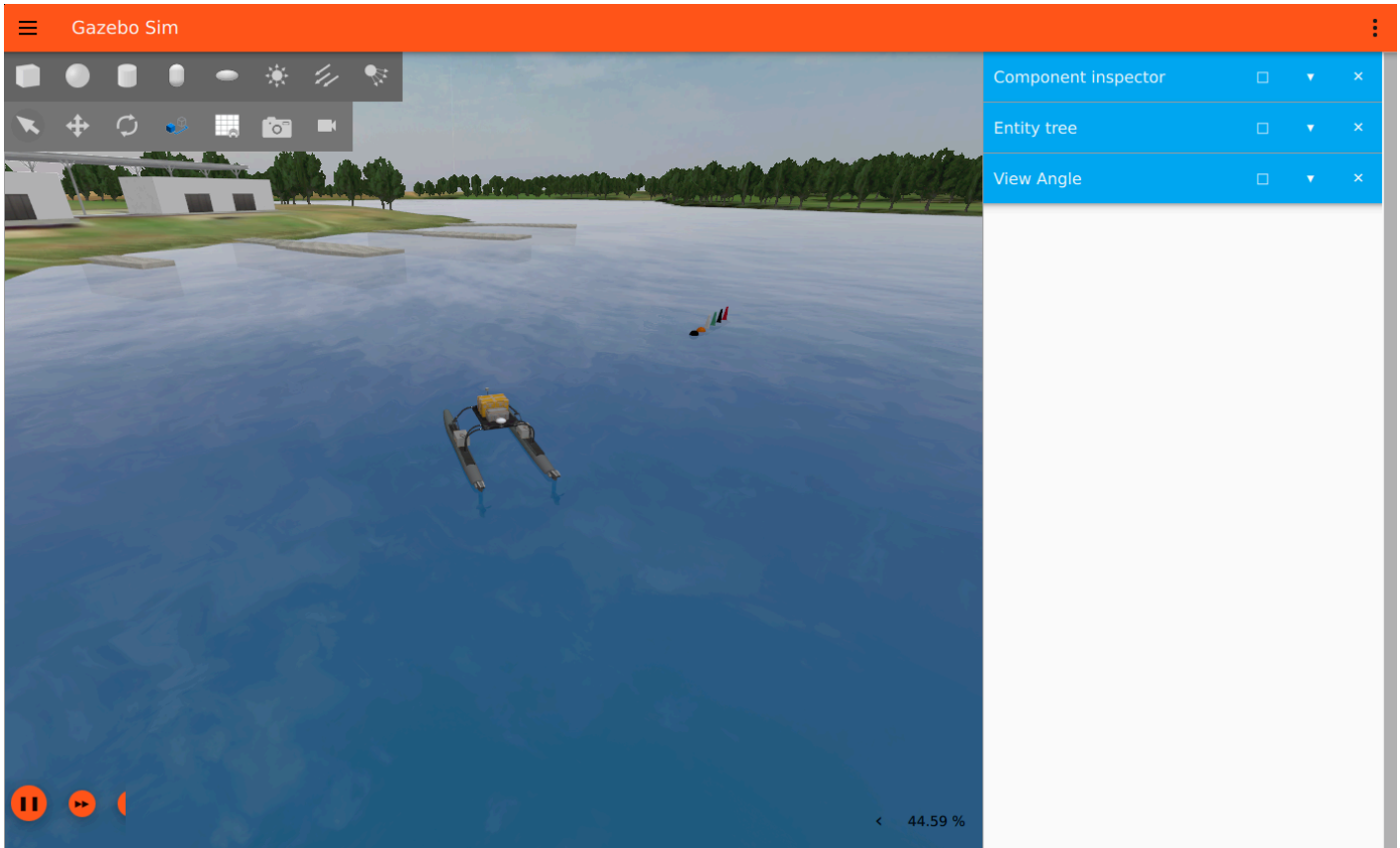
### 출발 상태



W 전진 10초 뒤



a 좌선회 8초 뒤



확인 항목	화면에서
배가 앞으로 나아간다	계류장이 멀어진다
선수 방향이 돌아간다	배가 카메라에 대해 비스듬해진다
오른쪽 아래 실시간 계수	35~50 % (데스크톱 기본) · 90 % (노트북 + ogre 옵션). 1 % 미만이면 §2-3 조치

#### ! 실시간 계수(RTF)가 100 % 가 아니어도 정상이다

파랑·부력 계산이 무겁다. 6주차 Simulink 연동에서 이 값을 직접 재서 페이싱을 맞춘다.

## 카메라를 배에 고정하기

- 손으로 마우스를 끌면 매번 화면이 달라진다. 명령으로 고정하면 재현된다

```
gz service -s /gui/follow --reqtype gz.msgs.StringMsg --reptype gz.msgs.Boolean \
--timeout 4000 --req 'data: "wamv"'
```

```
gz service -s /gui/follow/offset --reqtype gz.msgs.Vector3d --reptype gz.msgs.Boolean \
--timeout 4000 --req 'x: -14, y: 0, z: 7'
```

- 정상 출력

```
data: true
```

인자	뜻
x: -14	배 뒤쪽 14 m
y: 0	좌우 치우침 없음
z: 7	위로 7 m

## 추력 크기 바꾸기

- 실행 중에는 `+` / `-` 로 50 N 씩 조절한다
- 처음부터 다른 값으로 띄우려면

```
ros2 run usv_basics wamv_teleop_key --ros-args -p thrust:=400.0
```

- 다른 배(4주차에서 이름을 바꾼 경우)에 붙이려면

```
ros2 run usv_basics wamv_teleop_key --ros-args \
 -p left_topic:=/my_wamv/thrusters/left/thrust \
 -p right_topic:=/my_wamv/thrusters/right/thrust
```

## 관찰 과제 (필수) — 앞 절의 세 가지를 이 노드로 다시 본다

관찰	어떻게
1	<code>w</code> 로 가다가 <b>스페이스</b> . 몇 초, 몇 미터를 더 나아가는가
2	<code>a</code> 를 누른 <b>순간</b> 과 배가 돌기 시작하는 순간의 시간차
3	<code>w</code> 만 눌렀는데 옆으로 흐르는가 ( <code>ros2 topic echo /wamv/sensors/gps/gps/fix</code> 로 위치 확인)

★ 과제 3 에서 쓸 숫자가 여기서 나온다

눈으로만 보지 말고 `ros2 topic echo` 로 값을 받아 적을 것.

## 참고 — 다른 조종 방법 두 가지

방법	명령	언제
명령줄 직접 발행	<code>ros2 topic pub ...</code> (§2-5)	값 하나만 빠르게 시험할 때
키보드 노드	<code>ros2 run usv_basics wamv_teleop_key</code>	본 과목 기본
조이스틱	VRX 가 제공하는 <code>usv_joy_teleop.py</code>	게임패드가 있을 때

- VRX 쪽 조이스틱 스크립트의 위치는 아래에서 확인할 수 있다

```
ls ~/vrx_ws/src/vrx/vrx_gz/launch/
```

```
competition.launch.py spawn_config.launch.py spawn.launch.py
usv_joy_teleop.py vrx_environment.launch.py
```

- 게임패드가 없으면 쓸 수 없다. 수업에서는 키보드 노드를 쓴다

## 다음에 만날 월드 — 지금 미리 본다

- 지금까지 쓴 `sydney_regatta` 는 아무 과제도 없는 연습용 수면이다
- VRX 에는 **채점까지 되는 과제 월드**가 함께 들어 있다

```
ls ~/vrx_ws/src/vrx/vrx_gz/worlds/ | head
```

월드	본 과목에서
stationkeeping_task	10주차 동적위치유지
wayfinding_task	7주차 웨이포인트 유도
navigation_task	Term Project 1구간
scan_dock_deliver_task	Term Project 마지막 구간

- 자세한 내용과 채점 토픽은 4주차 §2-6 에서 다룬다

## 2-7. TF2 확인

- VRX가 실행 중인 상태에서

```
ros2 run tf2_tools view_frames
```

- 몇 초 기다린 뒤 자동 종료됨
- 현재 폴더에 frames\_\*.pdf 생성 → 열어서 트리 구조 확인

```
ros2 topic echo /tf_static --once
```

- 여기서 실제 프레임 이름을 확인한 뒤

```
ros2 run tf2_ros tf2_echo <상위프레임> <하위프레임>
```

## RViz2 로 보기

```
rviz2
```

- 좌하단 **Add** → **TF** 추가
- 좌측 **Global Options** → **Fixed Frame** 을 최상위 프레임으로 설정
- Add** → **PointCloud2** → Topic 을 LiDAR 토픽으로 지정
- 배를 움직이면서 프레임과 점군이 함께 움직이는지 확인

### RViz2 의 부하가 큰 경우

LiDAR 표시는 끄고 TF만 볼 것.

### 3부 · Simulink — 좌표 변환을 눈으로 확인한다

1부에서 쓴 변환식을 블록으로 만들어 돌려 본다. 모델 네 개가 한 단계씩 올라간다.

```
>> W03_setup % 기준점 · 시나리오 · 샘플링
>> build_w03_models % 모델 네 개를 다시 만든다 (깨졌을 때)
```

모델	무엇을 보는가	VRX 필요
W03_1_frame_check	변환식이 맞는지 숫자 하나로 확인	필요 없음
W03_2_vrx_nav	실제 센서를 받아 NED 로 바꾼다	필요
W03_3_vrx_drive	추력을 주고 부호를 확인한다	필요
W03_4_teleop	버튼으로 몰면서 궤적과 상태를 함께 본다	필요

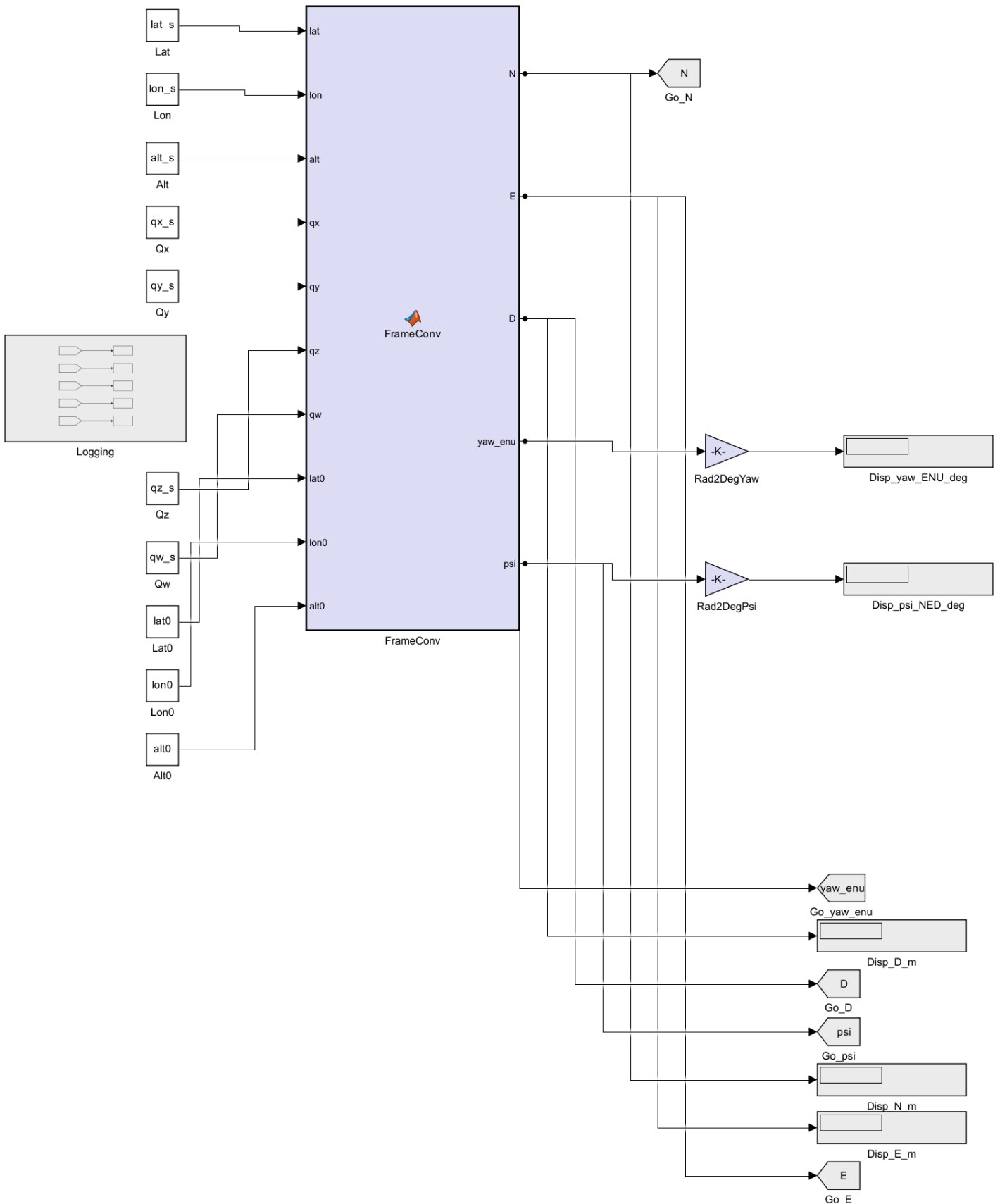
### 3-1. W03\_1\_frame\_check — VRX 없이 변환식만 계산

W03 1단계 — 좌표 변환식 확인 (VRX 불필요)

4주차 §2-3 에서 실제로 받은 GPS-IMU 값 한 발을 넣고 NED 로 바꾼다.

yaw\_ENU 와 psi\_NED 가 90도 차이인지, N-E 가 기준점에서 몇 m 인지 눈으로 확인한다.

실행 전 >> W03\_setup



- 상수 열 개(샘플 위경도·고도, 쿼터니언, 기준점)를 **FrameConv** 에 넣는다

- 출력은 **N**, **E**, **D**, **yaw\_enu**, **psi** 다섯 개
- ROS 도 Gazebo 도 필요 없다. 식이 맞는지 보는 것이 전부다

## 실행

```
>> W03_setup
>> sim('W03_1_frame_check')
```

## 실측 결과

값	결과	어떻게 확인하는가
<b>N</b>	<b>37.955</b> m	위도가 $3.42 \times 10^{-4}$ 도 북쪽 $\rightarrow \times R_M$
<b>E</b>	<b>-0.323</b> m	경도가 거의 같으므로 0 에 가깝다
<b>D</b>	<b>-0.064</b> m	고도가 기준보다 높으므로 음수
<b>yaw_enu</b>	<b>1.0001</b> rad = 57.30°	쿼터니언에서 뽑은 ENU 방위
<b>psi</b>	<b>0.5707</b> rad = 32.70°	$90^\circ - 57.30^\circ = 32.70^\circ$

★ 이 표의 마지막 두 줄이 이 모델의 전부다

$\psi_{NED} = 90^\circ - \psi_{ENU}$  가 숫자로 맞는지 본다.  
 $57.30 + 32.70 = 90.00$  이 되면 변환이 맞은 것이다.  
 손으로 계산해 보고 화면의 **Display** 와 맞춰 볼 것.

✗ **D** 가 음수인 것이 정상이다

NED 의 **D** 는 아래가 양수다. 배가 기준 고도보다 위에 있으면  $D < 0$  이다.  
 부호를 바꿔 쓰면 10주차 DP 에서 깊이 방향이 뒤집힌다.

## 해 볼 것

시도	관찰
<b>W03_setup</b> 의 <b>lat_s</b> 를 기준점과 같게	<b>N</b> 이 0 이 되는가
<b>lon_s</b> 를 $+10^{-4}$ 도 늘림	<b>E</b> 가 몇 미터 늘어나는가. 위도 쪽과 비교
<b>qz_s</b> , <b>qw_s</b> 를 (0,1) 로	<b>yaw_enu</b> = 0, <b>psi</b> = 90° 가 되는가

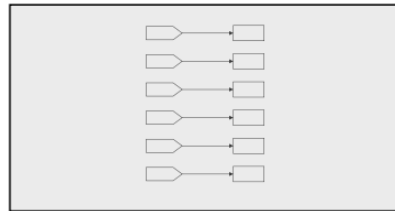
### 3-2. W03\_2\_vrx\_nav — 실제 센서를 NED 로

W03 2단계 — VRX 의 GPS·IMU 를 NED 로 (VRX 필요)

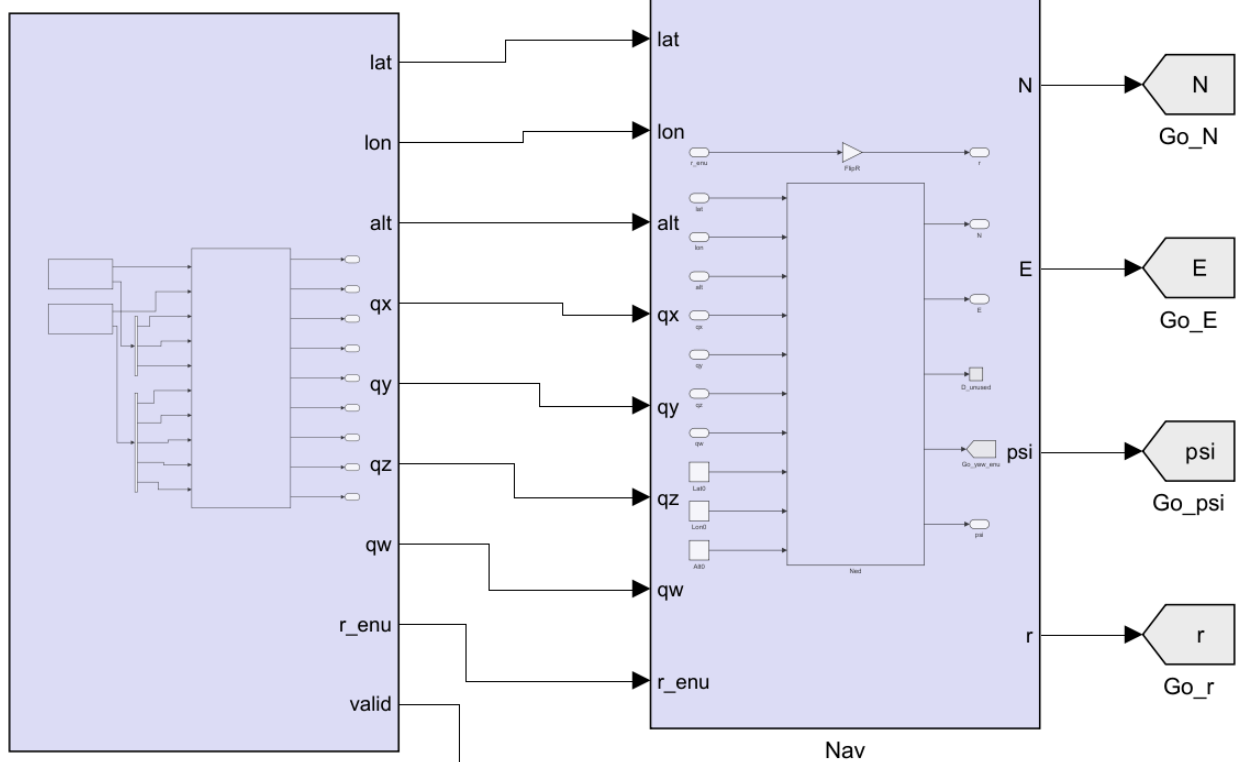
SensorSubscriber -> Nav -> [N] [E] [psi] [r] 태그

배를 손으로 몰면서(wamv\_teleop\_key) 숫자가 어떻게 바뀌는지 본다.

실행 전: (1) VRX 기동 (2) W03\_setup

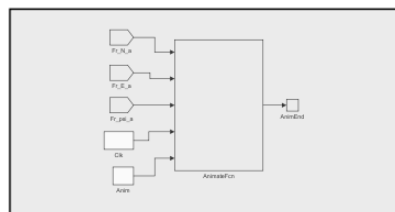


Logging



SensorSubscriber

Nav



Animate

- **SensorSubscriber** 가 GPS( **NavSatFix** )와 IMU( **Imu** )를 받는다

- `Nav` 가 1부의 변환식을 그대로 적용해 `N`, `E`, `psi`, `r` 을 낸다
- `Animate` 가 NED 평면에 항적을 실시간으로 그린다

### ★ RxLatch 가 있는 이유

`Subscribe` 블록은 첫 메시지가 오기 **전에도** 값을 낸다 — 전부 0 이다.  
 그런데 위도 0, 경도 0 은 기니만의 실제 좌표라서, 값만 보고는 "아직 안 왔다"를  
 구별할 수 없다. 그래서 `IsNew` 플래그를 걸어 두고 `valid` 로 내보낸다.

## 실행 순서

```
터미널 1 - VRX
ros2 launch vrx_gz competition.launch.py world:=sydney_regatta
```

```
% MATLAB
>> W03_setup
>> open_system('W03_2_vrx_nav') % Run 을 누른다
```

- `valid` 가 1 이 되어야 값이 의미가 있다. 0 이면 토픽이 오지 않는 것이다
- 배가 가만히 있으면 `N`, `E` 가 거의 변하지 않는다. 2-5 절 명령으로 밀어 본다

### 3-3. W03\_3\_vrx\_drive — 추력을 주고 부호를 확인한다

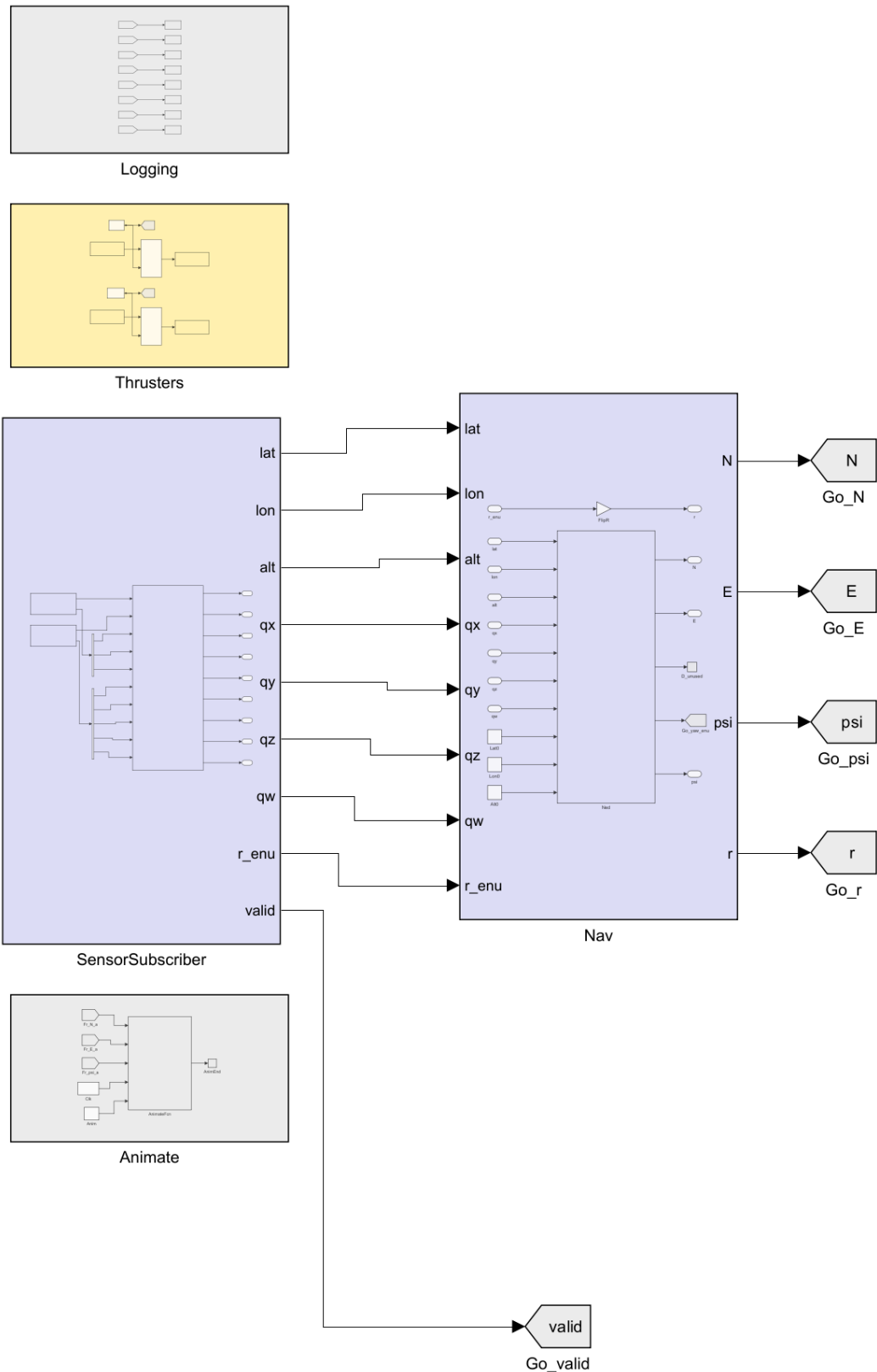
W03 3단계 — 추력을 주고 좌표 부호를 확인한다 (VRX 필요)

Thrusters 가 /wamv/thrusters/{left,right}/thrust 로 발행한다.

W03\_setup 의 scenario 를 1(직진) / 2(좌선회) 로 바꿔 두 번 돌린다.

좌선회에서 psi\_NED 가 줄고 r 이 음수여야 한다.

실행 전: (1) VRX 기동 (2) W03\_setup



- Thrusters 가 /wamv/thrusters/{left,right}/thrust 로 발행한다

- 추력은 `w03_setup` 의 `scenario` 로 고른다

scenario	좌	우	기대
1	+200 N	+200 N	직진. <code>psi</code> 가 거의 안 변함
2	-200 N	+200 N	좌선회. <code>psi</code> 가 줄고 <code>r</code> 이 음수

### ★ 이 모델의 목적은 속도가 아니라 부호다

좌선회에서 `psi_NED` 가 줄어드는지 본다. 늘어난다면 어딘가에서 부호가 한 번 더 뒤집힌 것이다 — 3주차 과제가 바로 이것을 수치로 증명하게 한다.

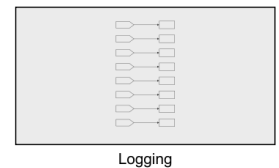
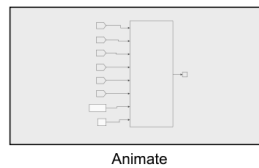
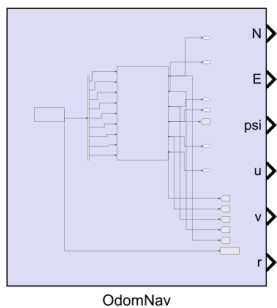
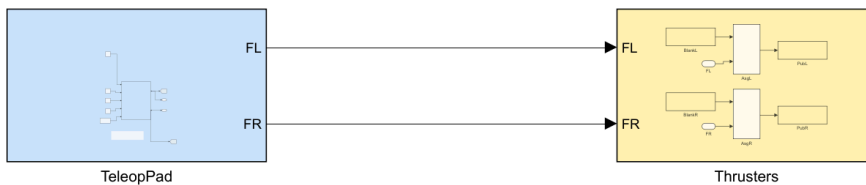
## 3-4. `w03_4_teleop` — 버튼으로 몰면서 상태를 본다

W03 4단계 — 버튼으로 배를 몰다 (VRX 필요)

`TeleopPad` 를 더블클릭하면 화살표 버튼 네 개가 나온다.  
누르고 있는 동안만 추력이 나간다 (Momentary).  
전진 / 후진 / 좌선회 / 우선회

ROS 2 노드 `wamv_teleop_key` 와 같은 일을 한다.  
다른 점은 눌린 값이 `Simulink` 안에 있어, 명령과 응답을 한 화면에서 함께 볼 수 있다는 것이다.

실행 전: (1) VRX 기동 (2) `w03_setup`



2-6 절의 `wamv_teleop_key` 와 같은 일을 하는 `Simulink` 판이다.

다른 점은 누른 값이 `Simulink` 안에 있어, 명령과 응답을 한 화면에서 함께 본다는 것이다.

### 버튼 — 신호가 아니라 파라미터를 누른다

`TeleopPad` 를 더블클릭하면 화살표 버튼 네 개가 나온다.

버튼	좌 추력	우 추력	결과
▲ 전진	$+F$	$+F$	앞으로
▼ 후진	$-F$	$-F$	뒤로
◀ 좌선회	$-F$	$+F$	제자리 좌선회
▶ 우선회	$+F$	$-F$	제자리 우선회

$$F_L = F, [(전진 - 후진) + (우 - 좌)], \quad F_R = F, [(전진 - 후진) - (우 - 좌)]$$

- $F$  는 `w03_setup` 의 `teleop_thrust` (기본 200 N)
- 전진과 선회를 **함께** 누르면 한쪽이 합쳐져 한계를 넘으므로 `Mix` 안에서 다시 자른다

### ★ Dashboard 버튼은 파라미터에 묶인다

일반 블록처럼 출력 포트가 있는 것이 아니다. 각 버튼이 옆에 있는 `Constant` 의 `Value` 에 묶여 있고, **누르는 동안만** 1 이 된다 (Momentary).  
그래서 **모델을 돌린 상태에서** 눌러야 반응한다.

## 화면 — 왼쪽은 어디 있는가, 오른쪽은 어떻게 움직이는가

`Animate` 가 창 하나를 띄운다.

자리	그리는 것
왼쪽	NED 평면의 항적. 가로축 East, 세로축 North (해도와 같은 방향)
오른쪽	위에서부터 $\psi, u, v, r$ 네 줄

### ★ 네 값이 움직이는 순서를 본다

좌선회 버튼을 누르면

1. 추력 차이가 요 모멘트를 만들어  $r$  이 먼저 선다
2.  $r$  이 쌓여  $\psi$  가 돈다 —  $\dot{\psi} = r$
3.  $\psi$  가 돌아야 항적이 된다

버튼을 떼면 역순으로 찾아든다. 1-4 절의 "힘 → 속도 → 위치" 가 이 세 줄이다.

### i $v$ 는 선회 중에만 0 이 아니다

직진 버튼만 누르면  $v \approx 0$  이다. 선회하면 배가 옆으로 밀려  $v$  가 생긴다.

1-5 절에서 본 크래프가 이때 나타난다.

## 실행 순서

```
터미널 1 - VRX
ros2 launch vrx_gz competition.launch.py world:=sydney_regatta
```

```
% MATLAB
>> W03_setup
>> open_system('W03_4_teleop')
```

1. **Run** 을 누른다 (정지 시간은 `T_end_teleop`, 기본 300 초)
2. `TeleopPad` 를 더블클릭해 버튼 창을 연다
3. 버튼을 눌러 가며 왼쪽 항적과 오른쪽 네 줄을 함께 본다

## 관찰 과제

시도	기록할 것
▲ 만 3초	$u$ 가 얼마까지 오르는가. $v, r$ 은 0 에 가까운가
◀ 만 3초	$r$ 이 먼저 서는가. $\psi$ 는 늘어나는가 <b>줄어드는가</b>
▲ 와 ◀ 를 함께	항적이 원호를 그리는가. $v$ 의 부호는
버튼을 떼 뒤	$u, r$ 이 0 으로 돌아오는 데 몇 초 걸리는가

**✕ 버튼을 떼면 추력이 0 이지만 배는 바로 서지 않는다**

물의 저항만으로 멈추므로 수 초가 걸린다. 이 **관성**이 6주차 이후 제어기가 상대할 대상이다.

# 마무리

## 이번 주차 요약

순서	한 일	확인 방법
1	Gazebo Garden 설치	<code>gz sim shapes.sdf</code>
2	VRX 내려받기 + 브랜치 변경	<code>git branch --show-current</code> → <code>humble</code>
3	VRX 빌드	<code>Summary: 5 packages finished</code>
4	WAM-V 스폰	시드니 레가타에 배가 떠 있음
5	토픽 조사	GPS / IMU / LiDAR 확인
6	명령어로 배 조종	직진 · 선회 성공
7	키보드로 배 조종	<code>wamv_teleop_key</code> 로 <code>w a s d</code> 동작
8	TF 트리 확인	<code>frames_*.pdf</code> 생성
9	Simulink 로 변환식 계산	<code>W03_1_frame_check</code> 에서 $57.30 + 32.70 = 90.00$
10	Simulink 버튼으로 배 조종	<code>W03_4_teleop</code> 에서 항적과 $\psi, u, v, r$ 을 함께 봄

## 수업 진도 체크

★ 다음 주 수업을 따라가기 위한 최소 조건

### 이론 이해

- ☐ SDF · URDF · Xacro 의 차이를 설명할 수 있다
- ☐ RTF 가 무엇이고 왜 확인해야 하는지 안다
- ☐ 선박 6자유도의 이름과 뜻을 말할 수 있다
- ☐ 왜 3자유도(Surge · Sway · Yaw)만 제어하는지 안다
- ☐ **ENU 와 NED 의 차이**를 그림으로 설명할 수 있다
- ☐ 좌표계 실수는 **에러가 나지 않는다**는 것을 안다
- ☐ 쿼터니언 순서가 ROS와 MATLAB에서 다르다는 것을 안다
- ☐ TF2 가 무엇을 해결하는지 설명할 수 있다

### 실습 완료

- ☐ `gz sim shapes.sdf` 실행 성공
- ☐ `git branch --show-current` 가 `humble` 출력 ← 반드시 확인
- ☐ `colcon build --merge-install` 성공
- ☐ `competition.launch.py world:=sydney_regatta` 로 WAM-V 스폰
- ☐ 파도가 움직이고 배가 흔들리는 것을 확인
- ☐ GPS / IMU 토픽을 `echo` , `hz` 로 확인
- ☐ LiDAR 토픽의 QoS `Reliability` 를 확인하고 적어 두었다
- ☐ `ros2 topic pub` 으로 배를 **직진**시켰다
- ☐ `ros2 topic pub` 으로 배를 **선회**시켰다

- ☐ `git pull` 로 `usv_basics` 를 갱신하고 빌드했다
- ☐ `ros2 pkg executables usv_basics` 에 `wamv_teleop_key` 가 보인다
- ☐ 키보드 `w a s d` 로 배를 몰았다
- ☐ `gz service` 로 카메라를 배에 고정해 봤다
- ☐ `q` 로 종료하면 배가 **선다**는 것을 확인했다
- ☐ `view_frames` 로 TF 트리 PDF 를 생성했다
- ☐ RViz2 에서 TF 를 표시했다
- ☐ `W03_setup` → `sim('W03_1_frame_check')` 로 `yaw_enu` + `psi` =  $90^\circ$  를 확인했다
- ☐ `W03_2_vrx_nav` 에서 `valid` 가 1 이 되는 것을 확인했다
- ☐ `W03_3_vrx_drive` 의 `scenario` 를 1:2 로 바꿔 `psi` 의 증감 방향을 확인했다
- ☐ `W03_4_teleop` 을 돌리고 버튼으로 배를 몰았다
- ☐ 좌선회 버튼에서 `r` 이 먼저 서고 그 다음 `psi` 가 도는 순서를 보았다

## 관찰 기록

- ☐ 추력을 끊은 뒤 미끄러지는 거리를 관찰했다
- ☐ 선회 명령의 응답 지연을 관찰했다
- ☐ 직진 명령에도 옆으로 흐르는 것을 관찰했다

## 과제 3 — 좌표 변환 노드

- 제출 기한: 4주차 수업 전
- 제출: 코드 + 검증 결과 + 짧은 분석

### ① `frame_converter` 노드 작성

- 위치: `~/capstone_ws/src/usv_basics/usv_basics/frame_converter.py`

#### 구독

- `/wamv/sensors/gps/gps/fix` (`sensor_msgs/NavSatFix`)
- `/wamv/sensors/imu/imu/data` (`sensor_msgs/Imu`)

#### 처리

1. GPS의 LLA를 **flat-earth** 근사로 NED 위치 (`N`, `E`, `D`) [m] 로 변환
  - 기준점 `lla0` 는 파라미터로 받거나, 첫 수신값을 기준으로 삼을 것
2. IMU 쿼터니언 → 오일러각 (`roll`, `pitch`, `yaw`) [rad]
  - 순서 (`x`, `y`, `z`, `w`) 주의
3. ENU 기준 yaw → **NED 기준 선수각** 으로 변환하고 -180 ~ +180 도로 정리

#### 발행

- `/usv/pose_ned` (`geometry_msgs/PoseStamped` 또는 자유 형식)
- 로그로 `N`, `E`, `roll`, `pitch`, `yaw_NED` 를 1 Hz 출력

### ② 검증 — 이 과제의 절반

#### ★ 주장은 수치로 검증한다

#### 검증 1 — 왕복 변환

- LLA → NED → LLA 했을 때 원래 값으로 돌아오는가?
- 오차를 **미터 단위로 보고** (1 mm 이하가 정상)

## 검증 2 — 방향 확인

조작	확인
배를 북쪽으로 직진	N 값이 증가하는가? 선수각이 0도에 가까운가?
배를 동쪽으로 직진	E 값이 증가하는가? 선수각이 +90도에 가까운가?

- 스크린샷 또는 로그로 제출

## 검증 3 — 각도 정리

- 배를 한 바퀴 이상 선회
- 선수각이 179도 → -180도 로 넘어가는 지점에서 값이 튀지 않는가?

### ③ 분석 (5~10줄)

1. ENU → NED 변환을 빠뜨렸다면 배는 어떻게 움직이겠는가? 구체적으로
2. 쿼터니언 순서를 혼동하면 어떤 증상이 나타나는가?
3. flat-earth 근사가 유효한 범위는 어느 정도이며, 넘으면 어떤 오차가 생기는가?

## 평가 기준

항목	배점
노드 정상 동작 (구독 · 변환 · 발행)	30%
검증 1~3 수행 및 수치 제시	40%
분석의 정확성	20%
코드 가독성 (변환 함수 분리, 좌표계 주석)	10%

### i 이번 학기 내내 이 기준으로 평가함

절반은 "동작하는 코드"가 아니라 "동작함을 증명한 기록" 임.

## 막혔을 때

증상	원인	해결
<code>colcon build</code> 중 프로세스가 죽음	메모리 부족	<code>--parallel-workers 2</code> , <code>.wslconfig</code> 메모리 상향
빌드는 됐는데 실행 시 플러그인 오류	브랜치 미지정	<code>cd ~/vrx_ws/src/vrx &amp;&amp; git checkout humble</code> 후 재빌드
Gazebo 창이 안 뜨고 멈춤	WSLg 미작동	<code>wsl --update</code> → <code>xeyes</code> 확인
Gazebo가 매우 느림 (RTF < 0.2)	GPU 미사용 / RAM 부족	다른 프로그램 종료, 워크스테이션 사용
RTF 가 1 % 미만 (창·배는 정상으로 보임)	WSL D3D12 경로의 카메라 센서 렌더링 병목 (Intel 그래픽에서 재현)	<code>"extra_gz_args:=--render-engine-server ogre"</code> 를 붙여 실행 (§2-3)
<code>ros2 topic list</code> 에 <code>wamv</code> 토픽이 없음	환경 미적용	<code>source ~/vrx_ws/install/setup.bash</code>
<code>ros2 topic pub</code> 했는데 배가 안 움직임	토픽명 불일치	<code>ros2 topic list   grep thrusters</code> 로 정확한 이름 확인
<code>view_frames</code> 가 빈 PDF 생성	TF 발행 노드 미실행	VRX 실행 확인, 몇 초 더 대기
모델 다운로드가 멈춤	네트워크	재시도. 캐시는 <code>~/ .gz/fuel/</code>

## 키보드 조종 ( `wamv_teleop_key` )

증상	원인	해결
<code>No executable found</code>	<code>git pull</code> 후 다시 빌드하지 않음	<code>cd ~/capstone_ws &amp;&amp; colcon build --symlink-install &amp;&amp; source install/setup.bash</code>
키를 눌러도 아무 반응 없음	터미널에 포커스가 없음	teleop 을 띄운 터미널 창을 클릭한 뒤 누른다
로그는 찍히는데 배가 안 움직임	토픽 이름 불일치	<code>ros2 topic list   grep thrust</code> 로 확인 후 <code>-p left_topic:=...</code> 로 지정
로그는 찍히는데 배가 안 움직임 (2)	VRX 가 일시정지 상태	Gazebo 왼쪽 아래 재생(▶) 버튼
창을 닫았는데 배가 계속 감	<code>q</code> 가 아니라 창을 강제로 닫음	<code>q</code> 로 종료한다. 이미 갔으면 <code>ros2 topic pub --once</code> 로 0 을 보낸다
배가 너무 느리다 / 빠르다	추력 기본값	실행 중 <code>+</code> / <code>-</code> , 또는 <code>-p thrust:=400.0</code>
<code>termios.error: (25, 'Inappropriate ioctl for device')</code>	터미널이 아닌 곳에서 실행 (파이프·스크립트)	터미널에서 직접 실행한다

## 참고 자료

### 이번 주차 실습 코드

- `usv_basics` 패키지 — [https://github.com/wkyouncnu/usv\\_basics](https://github.com/wkyouncnu/usv_basics)

- `wamv_teleop_key` — 이번 주차 §2-6 의 키보드 조종 노드
- 갠신은 `cd ~/capstone_ws/src/usv_basics && git pull` 후 재빌드
- `W03_simulink/` — 3부에서 쓰는 Simulink 모델과 스크립트

파일	하는 일
<code>W03_setup.m</code>	기준점 · 시나리오 · 샘플링 · <code>teleop_thrust</code>
<code>build_w03_models.m</code>	모델 네 개를 다시 만든다
<code>W03_1_frame_check.slx</code>	변환식 검산 (VRX 불필요)
<code>W03_2_vrx_nav.slx</code>	센서 → NED
<code>W03_3_vrx_drive.slx</code>	추력을 주고 부호 확인
<code>W03_4_teleop.slx</code>	버튼 조종 + 실시간 항적·상태
<code>W03_animate.m</code>	2·3단계의 항적 그림
<code>W03_teleop_plot.m</code>	4단계의 항적 + $\psi, u, v, r$ 그림
<code>W03_euler_order.m</code>	같은 세 각을 두 순서로 돌려 자세가 달라지는 것을 그린다 (1-6절)
<code>W03_vrx_run.m</code>	VRX 기동 · 정리 명령 모음

## 공식 문서

- Gazebo Garden — <https://gazebo-sim.org/docs/garden>
- VRX Wiki — <https://github.com/osrf/vrx/wiki>
- VRX 시스템 준비 — [https://github.com/osrf/vrx/wiki/preparing\\_system\\_tutorial](https://github.com/osrf/vrx/wiki/preparing_system_tutorial)
- ROS REP-103 (좌표계 표준) — <https://www.ros.org/reps/rep-0103.html>
- tf2 튜토리얼 — <https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Tf2-Main.html>
- URDF 튜토리얼 — <https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/URDF-Main.html>

## 볼트 내 문서

- [ENU와-NED를-섞으면-조용히-틀린다](#) — 이번 주차 이론의 핵심
- [배는-급정지하지-않는다](#) — 2-5 관찰 과제의 배경
- [WSL-VRX-환경구축](#) §4~§5 — 설치 절차
- [VRX-월드와-패키지](#) — 월드 목록과 토픽 정리

## 강의자료 폴더

- `2_지난 학기_강의 자료 /2025/11_동역학 모델 .pdf` — NED / Body 프레임, 운동학과 동역학의 구분. 이번 주차 1-4 ~ 1-6의 이론적 배경. 반드시 읽을 것

## 교재

- Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control*, 2nd ed. — Ch. 2 (Kinematics)
- Fossen, *Lecture Notes TTK4190* (NTNU, 무료 공개)
- MSS Toolbox — <https://github.com/cybergalactic/MSS>

## 다음 주 예고

- 4주차 — VRX 심화: 토픽 전수조사, WAM-V 모델 구조, Mapviz
- 할 일

- WAM-V의 URDF를 뜯어보고 센서를 직접 추가 · 이동
- 위성지도 위에 항적을 그리는 **Mapviz** 설정
- 팀 공용 문서가 될 **토픽 전수조사표** 완성
- 준비물
  - 이번 주차 완성한 VRX 환경
  - `git checkout humble` 을 확인하지 않은 학생은 반드시 먼저 확인